Table of Contents

1. Introduction
    1.1. Scope and Assumptions
2. How To Read This Document
3. Document Health Snapshot
4. Terms and Definitions
    4.1. Diagram Color and Shape Legend
5. Architecture Intent View
    5.1. What This View Answers
    5.2. View-Scoped Projection
    5.3. System Boundary Diagram
    5.4. System Decomposition Diagram
    5.5. Functional Group Manhattan Matrix
    5.6. Functional Unit Dependency Diagram
    5.7. Functional Groups and Units (View Scoped)
6. Communication View
    6.1. What This View Answers
    6.2. View-Scoped Projection
    6.3. Coverage Gaps
    6.4. Recommended Next Evidence Additions
    6.5. Communication Path Graph
    6.6. Functional Groups and Units (View Scoped)
7. Deployment View
    7.1. What This View Answers
    7.2. View-Scoped Projection
    7.3. Coverage Gaps
    7.4. Recommended Next Evidence Additions
    7.5. Deployment Artifact Relationship Graph
    7.6. Platform Operations Graph
    7.7. Functional Groups and Units (View Scoped)
8. Security View
    8.1. What This View Answers
    8.2. View-Scoped Projection
    8.3. Coverage Gaps
    8.4. Recommended Next Evidence Additions
    8.5. Threat Model Context Diagram
    8.6. Threat Model Data Flow Diagram
    8.7. Threat-to-Control Overlay Diagram
    8.8. Security Logging and Observability
    8.9. Attack Vector Chapters
9. Traceability View
    9.1. What This View Answers
    9.2. View-Scoped Projection
    9.3. Coverage Gaps

9.4. Recommended Next Evidence Additions
10. State Lifecycle View
10.1. What This View Answers
10.2. View-Scoped Projection
10.3. Coverage Gaps
10.4. Recommended Next Evidence Additions
10.5. Functional Groups and Units (View Scoped)
11. Interaction Flow View
11.1. What This View Answers
11.2. View-Scoped Projection
11.3. Coverage Gaps
11.4. Recommended Next Evidence Additions
11.5. Functional Groups and Units (View Scoped)
11.6. Requirement-to-Unit Mapping (Compact)
11.7. Verification Result Mapping
12. Traceability Appendix
12.1. Requirement Details
12.2. Verification Inventory
13. Reference Index
13.1. Authored References
13.2. Catalog References
13.3. Inferred Runtime References
13.4. Inferred Code References
13.5. Verification References

---

1. Introduction

This document describes the `payments` architecture using authored functional design and inferred runtime/code realization. It is intended to help product, `platform`, security, and implementation engineers reason about one shared model. Functional design is kept stable while realization details are inferred from infrastructure and source artifacts.

1.1. Scope and Assumptions

- `Functional Groups` and `Functional Units` are the stable authored design backbone.
- `Runtime elements` and `code elements` are inferred from available IaC, deployment, and source artifacts.
- Inferred coverage is evidence-driven and may be partial when source metadata is missing.
- `Requirements` and reference indexes are provided for traceability, not as primary narrative flow.

2. How To Read This Document

This document is intentionally split into two layers:

1. Concept layer: functional and view-specific architecture narratives for human understanding.
2. Evidence layer: generated requirement traceability and inferred reference indexes.

Read in this order:

1. Architecture Intent View: stable authored architecture intent and ownership boundaries.
2. Communication / Deployment / Security Views: viewpoint-specific operational behavior.
3. Traceability View: requirement mapping, evidence coverage, and confidence.
4. Reference Appendix: supporting reference IDs and inferred-source links.

3. Document Health Snapshot

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Architecture Intent View (`VIEW-ARCHITECTURE-INTENT`)	stable	Stable authored capability boundaries and ownership split; changes are intentional and infrequent.	6	6	0	high	Defaults authored units to covered; use authoredStatus as the primary publication signal.
Communication View (`VIEW-COMMUNICATION`)	in-review	Communication semantics are partially inferred and still being refined with explicit protocol and interaction metadata.	4	3	1	medium	Counts a unit as covered when direct inferred runtime evidence exists.
Deployment View (`VIEW-DEPLOYMENT`)	in-review	Deployment chain and control-plane mapping are mostly present, but ownership evidence remains incomplete for some units.	6	5	1	high	Counts a unit as covered when direct inferred runtime evidence exists.
Security View (`VIEW-SECURITY`)	draft	Threat mapping is present for key vectors, but control expectations and coverage breadth are still evolving.	4	4	0	high	Counts a unit as covered when explicit authored attack-vector mapping exists.

View	Authored Status	Authored Status Explanation	Units in Scope	Direct Evidence Coverage	Gap Count	Heuristic Confidence	Heuristic Basis
Traceability View (`VIEW-TRACEABILITY`)	in-review	Requirement-to-unit linkage is present; cross-layer ownership consistency still needs tightening.	6	5	4	high	Counts a unit as covered when direct inferred runtime or code evidence exists.
State Lifecycle View (`VIEW-STATE-LIFECYCLE`)	in-review	Lifecycle transitions and controls are explicitly modeled for payment decision progression.	0	0	0	n/a	Lifecycle heuristic is currently basic; treat authoredStatus as primary signal.
Interaction Flow View (`VIEW-INTERACTION-FLOW`)	in-review	Flow is authored for core checkout path and used to align UX/data movement semantics with architecture behavior.	4	4	0	high	Flow heuristic is step/branch-oriented; treat authoredStatus plus projection completeness as the primary signal.

4. Terms and Definitions

Term	Definition
`3DS verification is required` (COND-3DS-REQUIRED)	Condition indicating strong customer authentication must be performed. Aliases:3DS required
`actor` (EM-ACTOR)	A person or operational role that interacts with functional units.
`attack vector` (EM-ATTACK-VECTOR)	A technical misuse or attack path that targets functional, referenced, or runtime elements.
`authored mapping` (EM-AUTHORED-MAPPING)	An explicit relationship declared in architecture inputs between authored or referenced elements.
`bank gateway endpoint` (REF-BANK-GATEWAY-ENDPOINT)	External banking endpoint used for card authorization and decline decisions. Aliases:bank API

Term	Definition
`bank link is available` (STATE-BANK-LINK-UP)	Connectivity state showing bank gateway communication is currently healthy. Aliases:bank link up
`bank link is unavailable` (EVT-BANK-LINK-UNAVAILABLE)	Event raised when payment engine cannot reach the external bank gateway. Aliases:bank link down
`checkout handling` (FU-CHECKOUT)	Handles checkout session start and customer-facing payment messaging. Aliases:checkout
`checkout is active` (STATE-CHECKOUT-ACTIVE)	Processing state where the user is actively submitting or confirming payment. Aliases:checkout active
`cluster provisioning` (FU-CLUSTER-PROVISIONING)	Provides cluster-level platform setup and environment bootstrap. Aliases:day1 provisioning
`code element` (EM-CODE-ELEMENT)	An inferred code structure or ownership element discovered from source trees and build metadata.
`compromised dependency` (AV-COMPROMISED-DEPENDENCY)	Supply-chain compromise in runtime image or code dependency. Aliases:dependency compromise
`control` (EM-CONTROL)	An authored security, policy, or operational control used to constrain behavior and reduce risk.
`customer` (ACT-CUSTOMER)	End user submitting payment details during checkout. Aliases:buyer
`data object` (EM-DATA-OBJECT)	An authored data artifact or contract shape traced across flow, interface, and implementation boundaries.
`decline reason` (DATA-DECLINE-REASON)	Structured reason code or message explaining why authorization failed. Aliases:decline code, decline reasons
`deployment target` (EM-DEPLOYMENT-TARGET)	An authored deployment destination such as environment, cluster, namespace, or registry scope.
`event` (EM-EVENT)	An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes.
`flow` (EM-FLOW)	An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps.
`flow step` (EM-FLOW-STEP)	A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions.
`fraud check is enabled` (FEAT-FRAUD-CHECK)	Feature flag controlling whether risk scoring is required before authorization. Aliases:fraud check

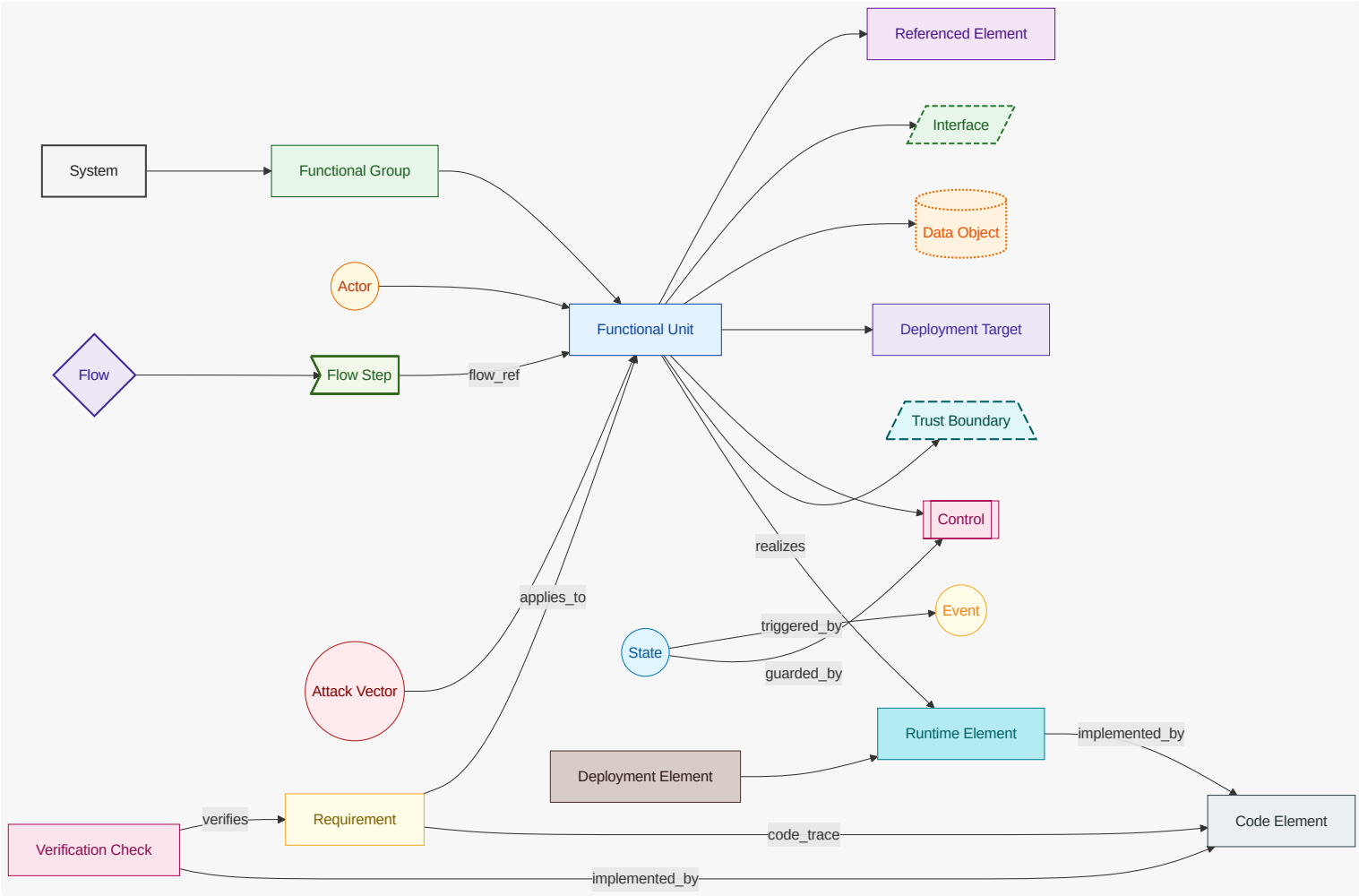
Term	Definition
`fraud detection system` (SYS-FRAUD-DETECTION-SYSTEM)	Fraud scoring and review subsystem participating in payment decisions. Aliases:the fraud detection system
`fraud evaluation` (FG-FRAUD)	Functional group for risk scoring and fraud review capabilities. Aliases:fraud domain
`fraudulent transaction pattern` (AV-FRAUDULENT-TRANSACTION-PATTERN)	Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals. Aliases:fraud abuse, payment fraud
`functional group` (EM-FUNCTIONAL-GROUP)	A major authored capability area that groups related functional units.
`functional unit` (EM-FUNCTIONAL-UNIT)	An authored working unit inside a functional group that owns specific behavior.
`gitops operations` (FU-GITOPS-OPERATIONS)	Operates declarative delivery and release management flows. Aliases:day2 operations
`helm platform` (REF-HELM-PLATFORM)	Runtime packaging and deployment substrate for workloads. Aliases:helm
`inferred mapping` (EM-INFERRED-MAPPING)	A discovered relationship that links inferred runtime/code elements upward to authored design.
`interface` (EM-INTERFACE)	An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit.
`malicious api request` (AV-MALICIOUS-API-REQUEST)	Crafted payload attempting to bypass validation or authorization logic. Aliases:malicious request
`normal mode is active` (MODE-NORMAL)	Standard operating mode with full external dependency usage enabled. Aliases:normal mode, standard operating mode
`payment authorization` (FU-PAYMENT-AUTHORIZATION)	Coordinates authorization flow and bank gateway interaction. Aliases:auth
`payment authorization is requested` (EVT-PAYMENT-AUTH-REQUESTED)	Event raised when checkout asks payment engine to authorize a transaction. Aliases:authorization requested, payment auth requested
`payment id` (DATA-PAYMENT-ID)	Stable identifier used to correlate a payment across services and records.
`payment is declined` (EVT-PAYMENT-DECLINED)	Event indicating authorization was rejected and customer decline handling must execute. Aliases:decline, payment declined
`payment review is pending` (STATE-REVIEW-PENDING)	Transaction state requiring manual support review before final disposition. Aliases:review pending, review state

Term	Definition
`payments` (FG-PAYMENTS)	Functional group for checkout and authorization capabilities. Aliases:payments domain
`payments system` (SYS-PAYMENTS-SYSTEM)	Payment processing system-of-interest covered by this architecture. Aliases:the payments system
`platform` (FG-PLATFORM)	Functional group for provisioning and platform operations capabilities. Aliases:platform engineering
`platform operator` (ACT-PLATFORM-OPERATOR)	Role responsible for GitOps and platform lifecycle operations. Aliases:platform ops
`postgresql client driver` (REF-POSTGRES-DRIVER)	Third-party library used for application data persistence connectivity. Aliases:postgres driver
`referenced element` (EM-REFERENCED-ELEMENT)	An architecture-relevant external, platform, or third-party dependency represented by role.
`replayed authorization callback` (AV-REPLAYED-AUTH-CALLBACK)	Replay of a valid callback to induce duplicate or unauthorized state transitions. Aliases:callback replay
`requirement` (EM-REQUIREMENT)	A structured requirement statement that defines expected system behavior and maps to functional ownership.
`risk score` (DATA-RISK-SCORE)	Numeric fraud score produced by risk evaluation logic.
`risk score is high` (EVT-RISK-HIGH)	Event emitted when computed fraud risk exceeds the configured high-risk threshold. Aliases:high risk, high-risk classification, high-risk classifications
`risk scoring` (FU-RISK-SCORING)	Evaluates transaction risk and returns score classification. Aliases:fraud scoring
`risk threshold is exceeded` (COND-RISK-THRESHOLD-EXCEEDED)	Condition indicating risk score crossed a policy threshold for escalation. Aliases:risk above threshold
`runtime element` (EM-RUNTIME-ELEMENT)	An inferred runtime realization element discovered from infrastructure and deployment sources.
`state` (EM-STATE)	An authored lifecycle state used to model transition behavior, guards, and event-driven progression.
`support agent` (ACT-SUPPORT)	Operations role that investigates declines and high-risk review cases. Aliases:support
`support agent is notified` (EVT-SUPPORT-NOTIFIED)	Event indicating manual review notification was sent to support operations. Aliases:support notified
`support review` (FU-SUPPORT-REVIEW)	Supports manual review and escalation handling for risky transactions. Aliases:manual review

Term	Definition
`trust boundary` (EM-TRUST-BOUNDARY)	An authored trust separation zone that marks policy, identity, or security control boundaries.
`upward linking` (EM-UPWARD-LINKING)	Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.
`verification check` (EM-VERIFICATION-CHECK)	An inferred or authored verification artifact that validates requirement behavior with test evidence.

4.1. Diagram Color and Shape Legend

This legend applies to all Mermaid diagrams in this document.



5. Architecture Intent View

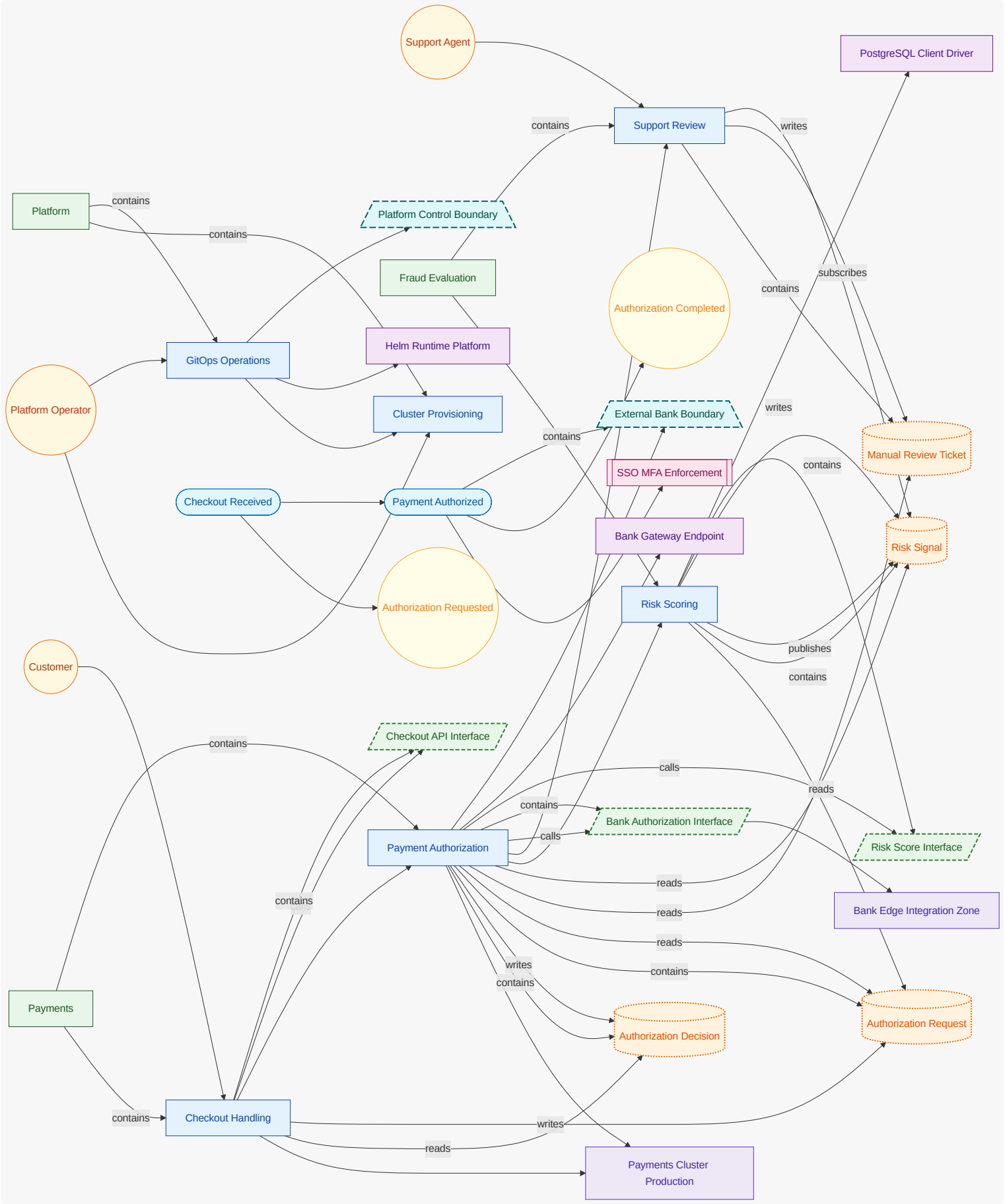
Show the stable authored architecture intent: capability areas, unit ownership boundaries, and intentional responsibility split.

5.1. What This View Answers

- What are the main capability areas?

- What are the stable units of responsibility and what does each unit own?
- How are responsibilities intentionally split?

5.2. View-Scoped Projection



Node ID	Kind	Label
ACT-CUSTOMER	actor	Customer
ACT-PLATFORM-OPERATOR	actor	Platform Operator
ACT-SUPPORT	actor	Support Agent
CTRL-PAYMENTS-SSO-MFA	control	SSO MFA Enforcement
DO-PAYMENTS-AUTH-DECISION	data_object	Authorization Decision
DO-PAYMENTS-AUTH-REQUEST	data_object	Authorization Request
DO-PAYMENTS-REVIEW-TICKET	data_object	Manual Review Ticket
DO-PAYMENTS-RISK-SIGNAL	data_object	Risk Signal
DEP-PAYMENTS-BANK-EDGE	deployment_target	Bank Edge Integration Zone
DEP-PAYMENTS-CLUSTER-PROD	deployment_target	Payments Cluster Production
EVT-PAYMENTS-AUTH-COMPLETED	event	Authorization Completed
EVT-PAYMENTS-AUTH-REQUESTED	event	Authorization Requested
FG-FRAUD	functional_group	Fraud Evaluation
FG-PAYMENTS	functional_group	Payments
FG-PLATFORM	functional_group	Platform
FU-CHECKOUT	functional_unit	Checkout Handling
FU-CLUSTER-PROVISIONING	functional_unit	Cluster Provisioning
FU-GITOPS-OPERATIONS	functional_unit	GitOps Operations

Node ID	Kind	Label
`FU-PAYMENT-AUTHORIZATION`	functional_unit	`Payment Authorization`
`FU-RISK-SCORING`	functional_unit	`Risk Scoring`
`FU-SUPPORT-REVIEW`	functional_unit	`Support Review`
`IF-PAYMENTS-BANK-AUTH`	`interface`	`Bank Authorization Interface`
`IF-PAYMENTS-CHECKOUT-API`	`interface`	`Checkout API Interface`
`IF-PAYMENTS-RISK-SCORE-API`	`interface`	`Risk Score Interface`
`REF-BANK-GATEWAY-ENDPOINT`	referenced_element	`Bank Gateway Endpoint`
`REF-HELM-PLATFORM`	referenced_element	`Helm` Runtime `Platform`
`REF-POSTGRES-DRIVER`	referenced_element	`PostgreSQL Client Driver`
`STATE-PAYMENTS-AUTHORIZED`	`state`	`Payment Authorized`
`STATE-PAYMENTS-CHECKOUT-RECEIVED`	`state`	`Checkout Received`
`TB-PAYMENTS-EXTERNAL-BANK`	trust_boundary	`External Bank Boundary`
`TB-PAYMENTS-PLATFORM-CONTROL`	trust_boundary	`Platform Control Boundary`

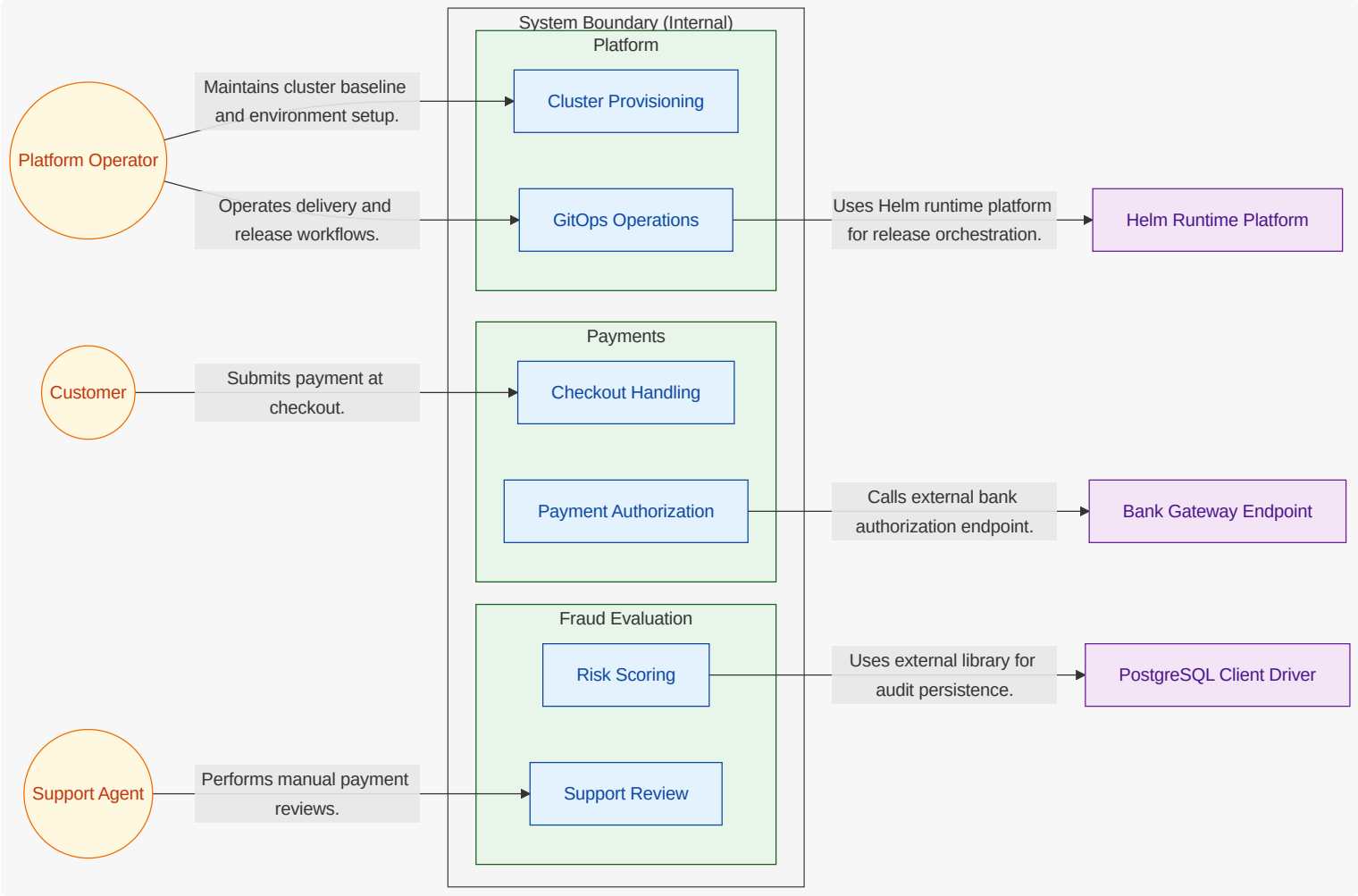
Mapping Type	From	To
bounded_by	`FU-GITOPS-OPERATIONS`	`TB-PAYMENTS-PLATFORM-CONTROL`
bounded_by	`FU-PAYMENT-AUTHORIZATION`	`TB-PAYMENTS-EXTERNAL-BANK`
calls	`FU-CHECKOUT`	`IF-PAYMENTS-CHECKOUT-API`

Mapping Type	From	To
calls	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-BANK-AUTH
calls	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-RISK-SCORE-API
contains	FG-FRAUD	FU-RISK-SCORING
contains	FG-FRAUD	FU-SUPPORT-REVIEW
contains	FG-PAYMENTS	FU-CHECKOUT
contains	FG-PAYMENTS	FU-PAYMENT-AUTHORIZATION
contains	FG-PLATFORM	FU-CLUSTER-PROVISIONING
contains	FG-PLATFORM	FU-GITOPS-OPERATIONS
contains	FU-CHECKOUT	IF-PAYMENTS-CHECKOUT-API
contains	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-DECISION
contains	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-REQUEST
contains	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-BANK-AUTH
contains	FU-RISK-SCORING	DO-PAYMENTS-RISK-SIGNAL
contains	FU-RISK-SCORING	IF-PAYMENTS-RISK-SCORE-API
contains	FU-SUPPORT-REVIEW	DO-PAYMENTS-REVIEW-TICKET
depends_on	FU-CHECKOUT	FU-PAYMENT-AUTHORIZATION
depends_on	FU-GITOPS-OPERATIONS	FU-CLUSTER-PROVISIONING
depends_on	FU-GITOPS-OPERATIONS	REF-HELM-PLATFORM
depends_on	FU-PAYMENT-AUTHORIZATION	FU-RISK-SCORING
depends_on	FU-PAYMENT-AUTHORIZATION	FU-SUPPORT-REVIEW
depends_on	FU-PAYMENT-AUTHORIZATION	REF-BANK-GATEWAY-ENDPOINT
depends_on	FU-RISK-SCORING	REF-POSTGRES-DRIVER
deployed_to	FU-CHECKOUT	DEP-PAYMENTS-CLUSTER-PROD
deployed_to	FU-PAYMENT-AUTHORIZATION	DEP-PAYMENTS-CLUSTER-PROD
deployed_to	IF-PAYMENTS-BANK-AUTH	DEP-PAYMENTS-BANK-EDGE

Mapping Type	From	To
guarded_by	STATE-PAYMENTS-AUTHORIZED	CTRL-PAYMENTS-SSO-MFA
guarded_by	STATE-PAYMENTS-AUTHORIZED	TB-PAYMENTS-EXTERNAL-BANK
interacts_with	ACT-CUSTOMER	FU-CHECKOUT
interacts_with	ACT-PLATFORM-OPERATOR	FU-CLUSTER-PROVISIONING
interacts_with	ACT-PLATFORM-OPERATOR	FU-GITOPS-OPERATIONS
interacts_with	ACT-SUPPORT	FU-SUPPORT-REVIEW
publishes	FU-RISK-SCORING	DO-PAYMENTS-RISK-SIGNAL
reads	FU-CHECKOUT	DO-PAYMENTS-AUTH-DECISION
reads	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-REQUEST
reads	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-REVIEW-TICKET
reads	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-RISK-SIGNAL
reads	FU-RISK-SCORING	DO-PAYMENTS-AUTH-REQUEST
subscribes	FU-SUPPORT-REVIEW	DO-PAYMENTS-RISK-SIGNAL
transitions_to	STATE-PAYMENTS-CHECKOUT-RECEIVED	STATE-PAYMENTS-AUTHORIZED
triggered_by	STATE-PAYMENTS-AUTHORIZED	EVT-PAYMENTS-AUTH-COMPLETED
triggered_by	STATE-PAYMENTS-CHECKOUT-RECEIVED	EVT-PAYMENTS-AUTH-REQUESTED
writes	FU-CHECKOUT	DO-PAYMENTS-AUTH-REQUEST
writes	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-DECISION
writes	FU-RISK-SCORING	DO-PAYMENTS-RISK-SIGNAL
writes	FU-SUPPORT-REVIEW	DO-PAYMENTS-REVIEW-TICKET

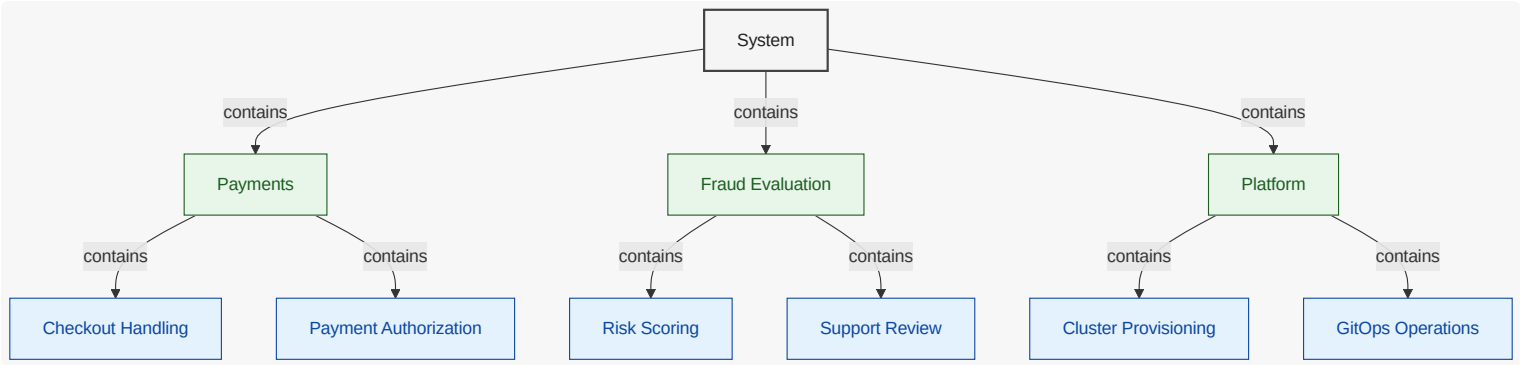
5.3. System Boundary Diagram

What this diagram shows: explicit internal vs external scope. Functional units are inside the `System Boundary (Internal)` container, while actors and referenced dependencies are shown outside the boundary.



5.4. System Decomposition Diagram

What this diagram shows: stable capability decomposition from functional groups into functional units.



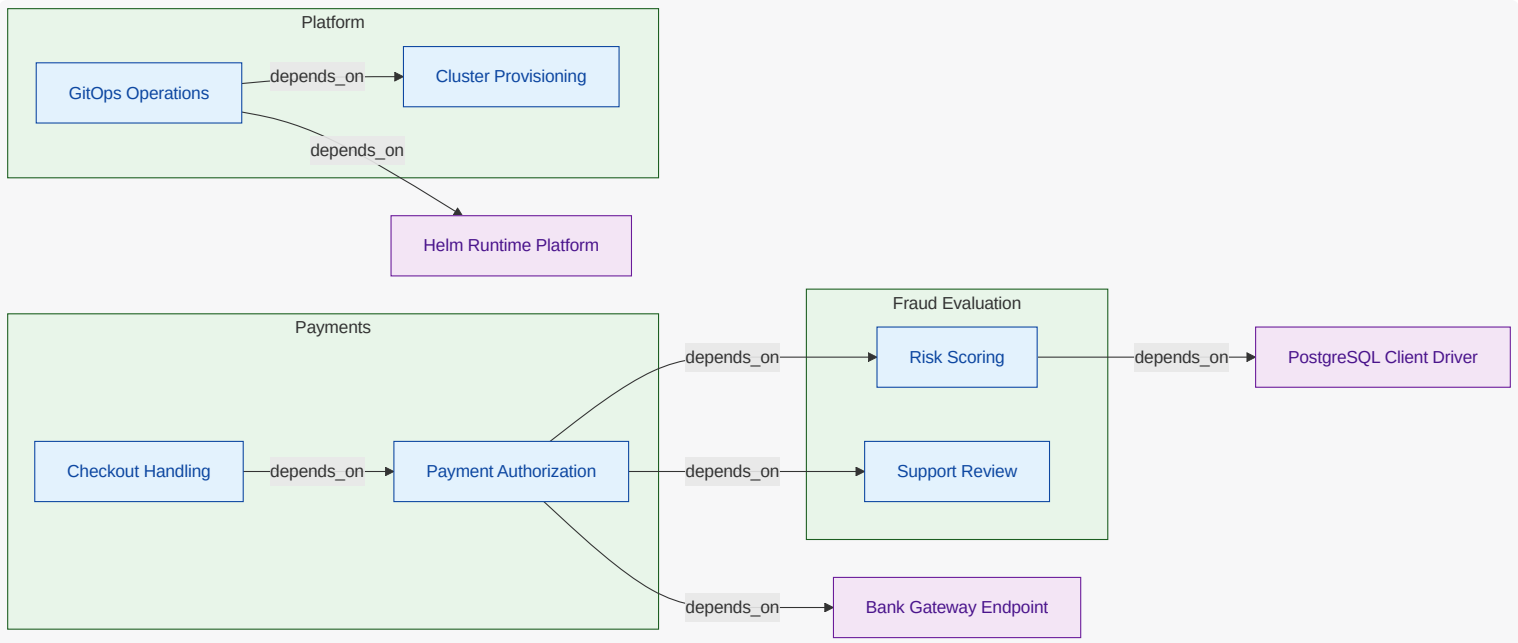
5.5. Functional Group Manhattan Matrix

What this diagram shows: a table-like matrix where functional groups are the bottom row (columns) and functional units are arranged in rows above their owning group.

Checkout Handling	Risk Scoring	Cluster Provisioning
Payment Authorization	Support Review	GitOps Operations
Payments	Fraud Evaluation	Platform

5.6. Functional Unit Dependency Diagram

What this diagram shows: dependency paths between functional units (grouped in Functional Group containers) and key referenced elements. Arrow meaning: `A -→|depends_on| B` means unit `A` requires behavior or data from `B` to fulfill its responsibility.



5.7. Functional Groups and Units (View Scoped)

5.7.1. Payments (FG-PAYMENTS)

Core payment `checkout` and authorization domain.

The `payments` group defines the product-facing payment journey from user intent to final payment outcome. It owns the consistency of `customer` messaging and transaction decisions when a payment is attempted, declined, retried, or temporarily unavailable. The group keeps `customer` `flow` stable while depending on fraud and external bank capabilities.

Checkout Handling (FU-CHECKOUT)

`Checkout handling` is the user-facing entrypoint where payment requests are initiated and normalized. It validates request shape, preserves transaction context, and returns clear `customer` feedback for each outcome. It delegates decision logic to `payment authorization` while protecting the user experience boundary.

Owned Responsibility

functional responsibility for `checkout handling`; includes decision and orchestration `flow` to `Payment Authorization`

Key Collaborators

`Payment Authorization`

Linked Requirements

`REQ-PAY-001`, `REQ-PAY-003`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-008`

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`Payment authorization` orchestrates the final transaction decision path. It coordinates `fraud scoring`, external bank interactions, and escalation to `support review` when needed. The unit returns deterministic outcomes so downstream behavior remains consistent and testable.

Owned Responsibility

functional responsibility for `payment authorization`; includes decision and orchestration `flow` to `Bank Gateway Endpoint` and decision and orchestration `flow` to `Risk Scoring`

Key Collaborators

`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`

Linked Requirements

`REQ-PAY-001`, `REQ-PAY-004`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-009`

5.7.2. Fraud Evaluation (FG-FRAUD)

`Risk scoring` and fraud audit domain.

The `fraud evaluation` group exists to reduce unsafe approvals while preserving throughput for legitimate `payments`. It combines automated `risk scoring` with manual `support review` so uncertain transactions move through a controlled escalation path. The group ensures `risk` decisions are explainable, auditable, and operationally actionable.

Risk Scoring (FU-RISK-SCORING)

`Risk scoring` computes and classifies transaction `risk` before approval decisions are finalized. It provides a stable scoring contract to authorization and supports audit context for later analysis. The unit is focused on decision quality, consistency, and policy-driven classification behavior.

Owned Responsibility

functional responsibility for `risk scoring`; includes decision and orchestration `flow` to `PostgreSQL Client Driver`

Key Collaborators

`PostgreSQL Client Driver`

Linked Requirements

`REQ-PAY-002`, `REQ-PAY-005`

Support Review (FU-SUPPORT-REVIEW)

`Support review` handles manual decisions for escalated or ambiguous payment cases. It gives `support` operators context to approve, reject, or request additional verification in a controlled `flow`. The unit ensures manual intervention remains auditable, policy-constrained, and operationally reliable.

Owned Responsibility

functional responsibility for `support review`

Key Collaborators

none

Linked Requirements

`REQ-PAY-004`, `REQ-PAY-009`

5.7.3. Platform (FG-PLATFORM)

`Cluster provisioning` and `GitOps operations` domain.

The `platform` group provides the runtime and deployment foundation used by all product capabilities. It owns environment provisioning, namespace and cluster boundaries, and continuous reconciliation behavior. The group enables safe delivery and reliable operations without owning payment business logic.

Cluster Provisioning (FU-CLUSTER-PROVISIONING)

`Cluster provisioning` creates the runtime substrate used by all application workloads. It establishes cluster and namespace structure, baseline controls, and environment-level readiness. This unit is responsible for predictable, repeatable infrastructure foundations.

Owned Responsibility

functional responsibility for `cluster provisioning`

Key Collaborators

none

Linked Requirements

no explicit authored `requirement` trigger

GitOps Operations (FU-GITOPS-OPERATIONS)

`GitOps operations` continuously reconciles intended release `state` with actual runtime `state`. It governs rollout `flow`, drift correction, and operational delivery confidence across payment and fraud workloads. The unit provides safe and observable release behavior as an ongoing operational capability.

Owned Responsibility

functional responsibility for `gitops operations`; includes decision and orchestration `flow` to `Cluster Provisioning` and decision and orchestration `flow` to `Helm` Runtime `Platform`

Key Collaborators

`Cluster Provisioning`, `Helm` Runtime `Platform`

Linked Requirements

no explicit authored `requirement` trigger

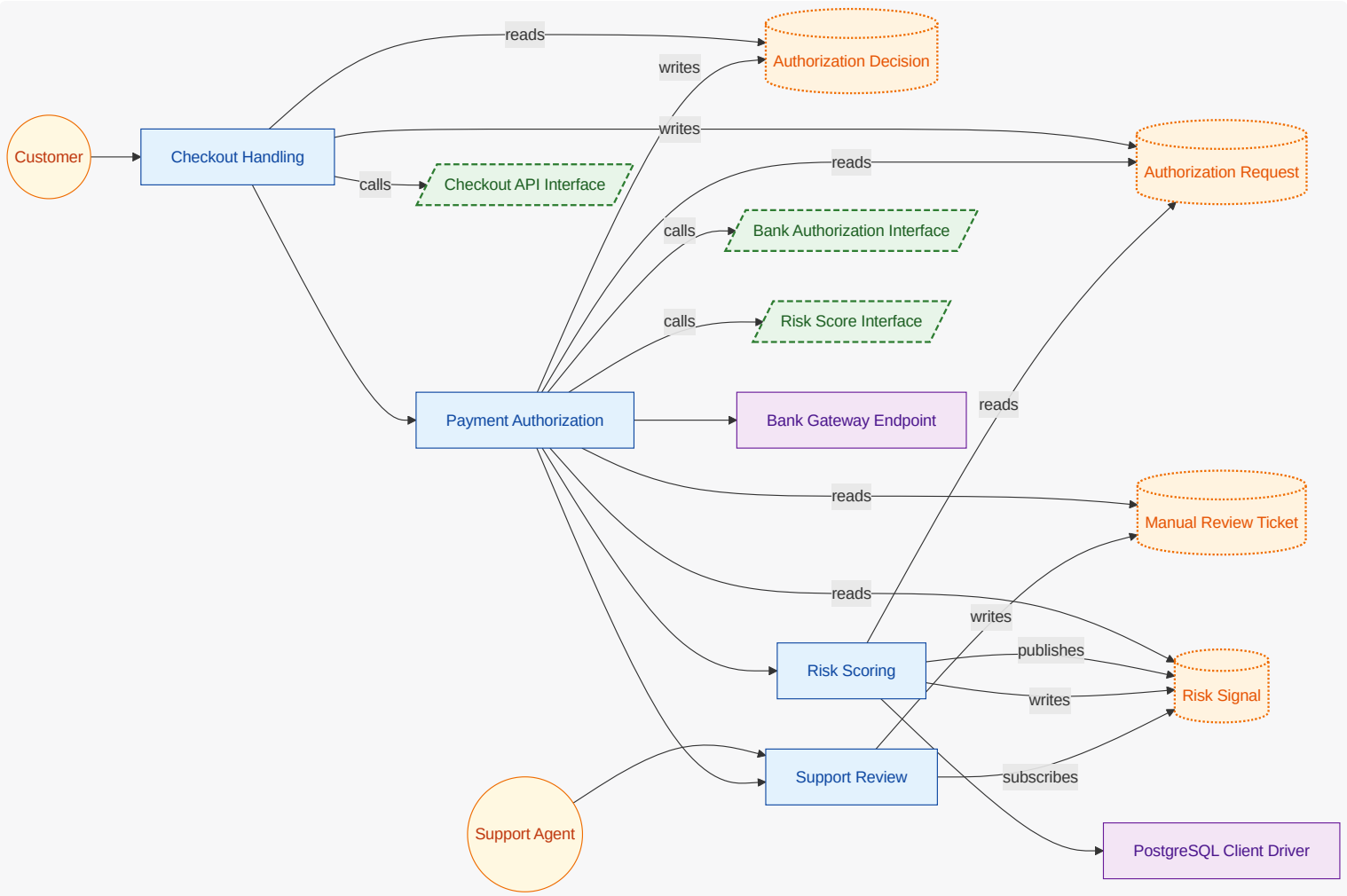
6. Communication View

Show who communicates with whom across key request paths, including `interface` edges and dependency handoffs.

6.1. What This View Answers

- Who talks to whom and over which interfaces?
- Which edges are synchronous, asynchronous, event-driven, or callback-oriented?
- Which communication paths are critical to the main request `flow`?

6.2. View-Scoped Projection



Node ID	Kind	Label
`ACT-CUSTOMER`	`actor`	`Customer`
`ACT-SUPPORT`	`actor`	`Support Agent`
`DO-PAYMENTS-AUTH-DECISION`	data_object	`Authorization Decision`

Node ID	Kind	Label
`DO-PAYMENTS-AUTH-REQUEST`	data_object	`Authorization Request`
`DO-PAYMENTS-REVIEW-TICKET`	data_object	`Manual Review Ticket`
`DO-PAYMENTS-RISK-SIGNAL`	data_object	`Risk Signal`
`FU-CHECKOUT`	functional_unit	`Checkout Handling`
`FU-PAYMENT-AUTHORIZATION`	functional_unit	`Payment Authorization`
`FU-RISK-SCORING`	functional_unit	`Risk Scoring`
`FU-SUPPORT-REVIEW`	functional_unit	`Support Review`
`IF-PAYMENTS-BANK-AUTH`	`interface`	`Bank Authorization Interface`
`IF-PAYMENTS-CHECKOUT-API`	`interface`	`Checkout API Interface`
`IF-PAYMENTS-RISK-SCORE-API`	`interface`	`Risk Score Interface`
`REF-BANK-GATEWAY-ENDPOINT`	referenced_element	`Bank Gateway Endpoint`
`REF-POSTGRES-DRIVER`	referenced_element	`PostgreSQL Client Driver`

Mapping Type	From	To
calls	`FU-CHECKOUT`	`IF-PAYMENTS-CHECKOUT-API`
calls	`FU-PAYMENT-AUTHORIZATION`	`IF-PAYMENTS-BANK-AUTH`
calls	`FU-PAYMENT-AUTHORIZATION`	`IF-PAYMENTS-RISK-SCORE-API`
depends_on	`FU-CHECKOUT`	`FU-PAYMENT-AUTHORIZATION`
depends_on	`FU-PAYMENT-AUTHORIZATION`	`FU-RISK-SCORING`
depends_on	`FU-PAYMENT-AUTHORIZATION`	`FU-SUPPORT-REVIEW`

Mapping Type	From	To
depends_on	`FU-PAYMENT-AUTHORIZATION`	`REF-BANK-GATEWAY-ENDPOINT`
depends_on	`FU-RISK-SCORING`	`REF-POSTGRES-DRIVER`
interacts_with	`ACT-CUSTOMER`	`FU-CHECKOUT`
interacts_with	`ACT-SUPPORT`	`FU-SUPPORT-REVIEW`
publishes	`FU-RISK-SCORING`	`DO-PAYMENTS-RISK-SIGNAL`
reads	`FU-CHECKOUT`	`DO-PAYMENTS-AUTH-DECISION`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-AUTH-REQUEST`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-REVIEW-TICKET`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-RISK-SIGNAL`
reads	`FU-RISK-SCORING`	`DO-PAYMENTS-AUTH-REQUEST`
subscribes	`FU-SUPPORT-REVIEW`	`DO-PAYMENTS-RISK-SIGNAL`
writes	`FU-CHECKOUT`	`DO-PAYMENTS-AUTH-REQUEST`
writes	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-AUTH-DECISION`
writes	`FU-RISK-SCORING`	`DO-PAYMENTS-RISK-SIGNAL`
writes	`FU-SUPPORT-REVIEW`	`DO-PAYMENTS-REVIEW-TICKET`

6.3. Coverage Gaps

Coverage summary: 3/4 units have direct evidence coverage in this view.

- `Support Review` has no direct inferred runtime/deployment evidence yet.

6.4. Recommended Next Evidence Additions

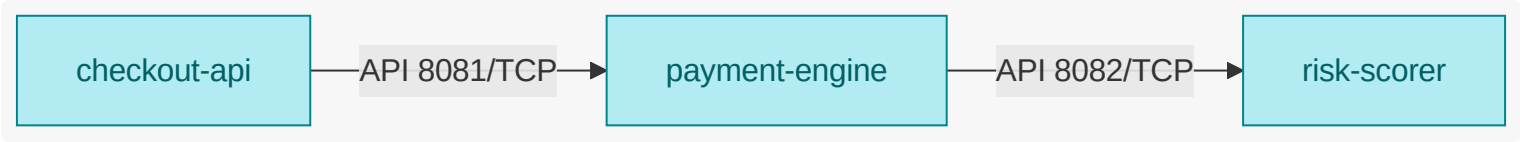
- Add explicit communication metadata (protocol, sync/async/`event`/callback) to `authored mappings` where currently implicit.
- Expose service protocol/port metadata in `Helm` or manifest values to enrich interface-edge inference.
- Capture missing communicating runtime units by adding discoverable workload artifacts under scanned inference roots.

6.5. Communication Path Graph

What this diagram shows: inferred unit-to-unit request/call flow, including available interface/port labels.

NOTE

This graph is inferred from available deployment/runtime artifacts and may be partial. Missing edges or interface details indicate missing derived communication evidence, not necessarily missing behavior.



6.6. Functional Groups and Units (View Scoped)

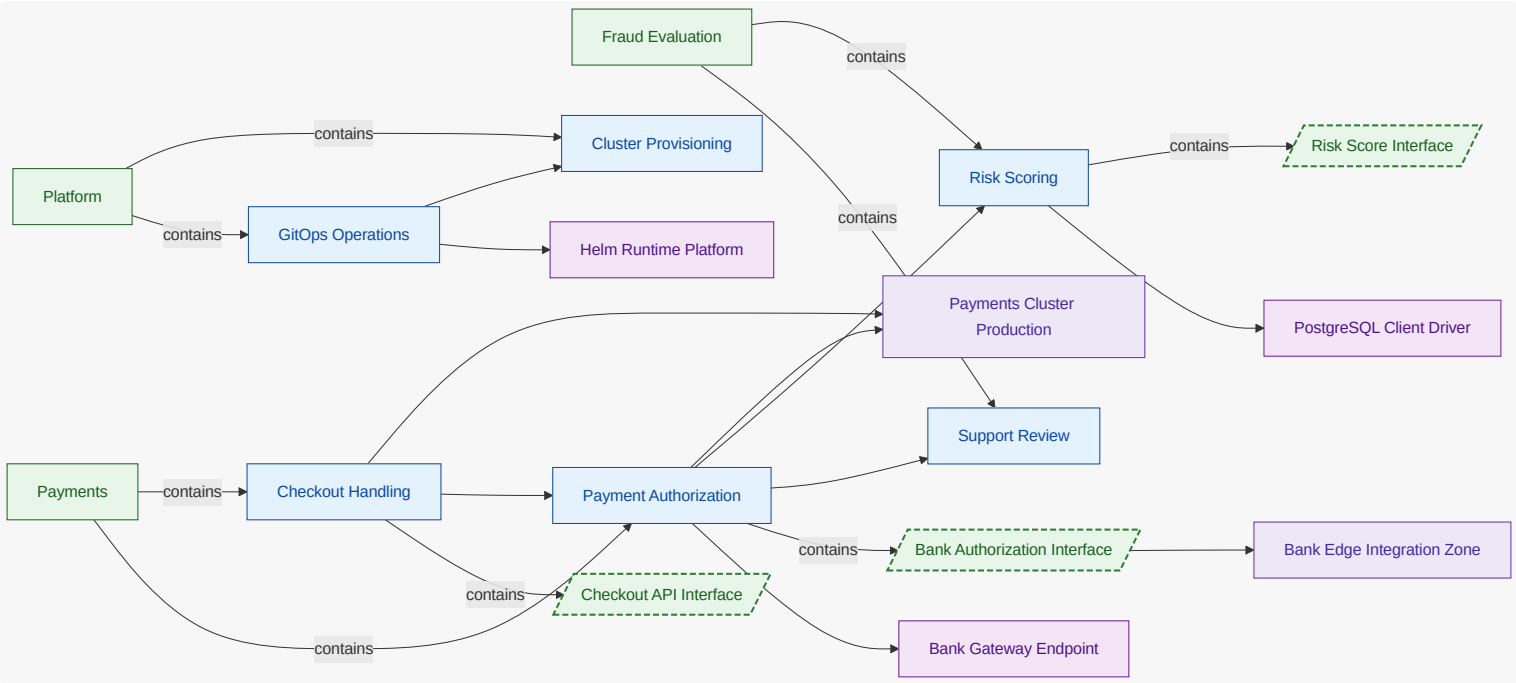
7. Deployment View

Show inferred deployment artifacts and ownership mapping to authored units. This view emphasizes deployment relationships and `platform` operations.

7.1. What This View Answers

- How does authored intent become deployed runtime?
- What control-plane objects manage deployable units?
- Where are `platform` ownership boundaries and missing ownership signals?

7.2. View-Scoped Projection



Node ID	Kind	Label
`DEP-PAYMENTS-BANK-EDGE`	deployment_target	`Bank Edge Integration Zone`
`DEP-PAYMENTS-CLUSTER-PROD`	deployment_target	`Payments Cluster Production`
`FG-FRAUD`	functional_group	`Fraud Evaluation`
`FG-PAYMENTS`	functional_group	`Payments`
`FG-PLATFORM`	functional_group	`Platform`
`FU-CHECKOUT`	functional_unit	`Checkout Handling`

Node ID	Kind	Label
`FU-CLUSTER-PROVISIONING`	functional_unit	`Cluster Provisioning`
`FU-GITOPS-OPERATIONS`	functional_unit	`GitOps Operations`
`FU-PAYMENT-AUTHORIZATION`	functional_unit	`Payment Authorization`
`FU-RISK-SCORING`	functional_unit	`Risk Scoring`
`FU-SUPPORT-REVIEW`	functional_unit	`Support Review`
`IF-PAYMENTS-BANK-AUTH`	`interface`	`Bank Authorization Interface`
`IF-PAYMENTS-CHECKOUT-API`	`interface`	`Checkout API Interface`
`IF-PAYMENTS-RISK-SCORE-API`	`interface`	`Risk Score Interface`
`REF-BANK-GATEWAY-ENDPOINT`	referenced_element	`Bank Gateway Endpoint`
`REF-HELM-PLATFORM`	referenced_element	`Helm` Runtime `Platform`
`REF-POSTGRES-DRIVER`	referenced_element	`PostgreSQL Client Driver`

Mapping Type	From	To
contains	`FG-FRAUD`	`FU-RISK-SCORING`
contains	`FG-FRAUD`	`FU-SUPPORT-REVIEW`
contains	`FG-PAYMENTS`	`FU-CHECKOUT`
contains	`FG-PAYMENTS`	`FU-PAYMENT-AUTHORIZATION`
contains	`FG-PLATFORM`	`FU-CLUSTER-PROVISIONING`
contains	`FG-PLATFORM`	`FU-GITOPS-OPERATIONS`
contains	`FU-CHECKOUT`	`IF-PAYMENTS-CHECKOUT-API`

Mapping Type	From	To
contains	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-BANK-AUTH
contains	FU-RISK-SCORING	IF-PAYMENTS-RISK-SCORE-API
depends_on	FU-CHECKOUT	FU-PAYMENT-AUTHORIZATION
depends_on	FU-GITOPS-OPERATIONS	FU-CLUSTER-PROVISIONING
depends_on	FU-GITOPS-OPERATIONS	REF-HELM-PLATFORM
depends_on	FU-PAYMENT-AUTHORIZATION	FU-RISK-SCORING
depends_on	FU-PAYMENT-AUTHORIZATION	FU-SUPPORT-REVIEW
depends_on	FU-PAYMENT-AUTHORIZATION	REF-BANK-GATEWAY-ENDPOINT
depends_on	FU-RISK-SCORING	REF-POSTGRES-DRIVER
deployed_to	FU-CHECKOUT	DEP-PAYMENTS-CLUSTER-PROD
deployed_to	FU-PAYMENT-AUTHORIZATION	DEP-PAYMENTS-CLUSTER-PROD
deployed_to	IF-PAYMENTS-BANK-AUTH	DEP-PAYMENTS-BANK-EDGE

7.3. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- `Support Review` has no direct inferred runtime/deployment evidence yet.

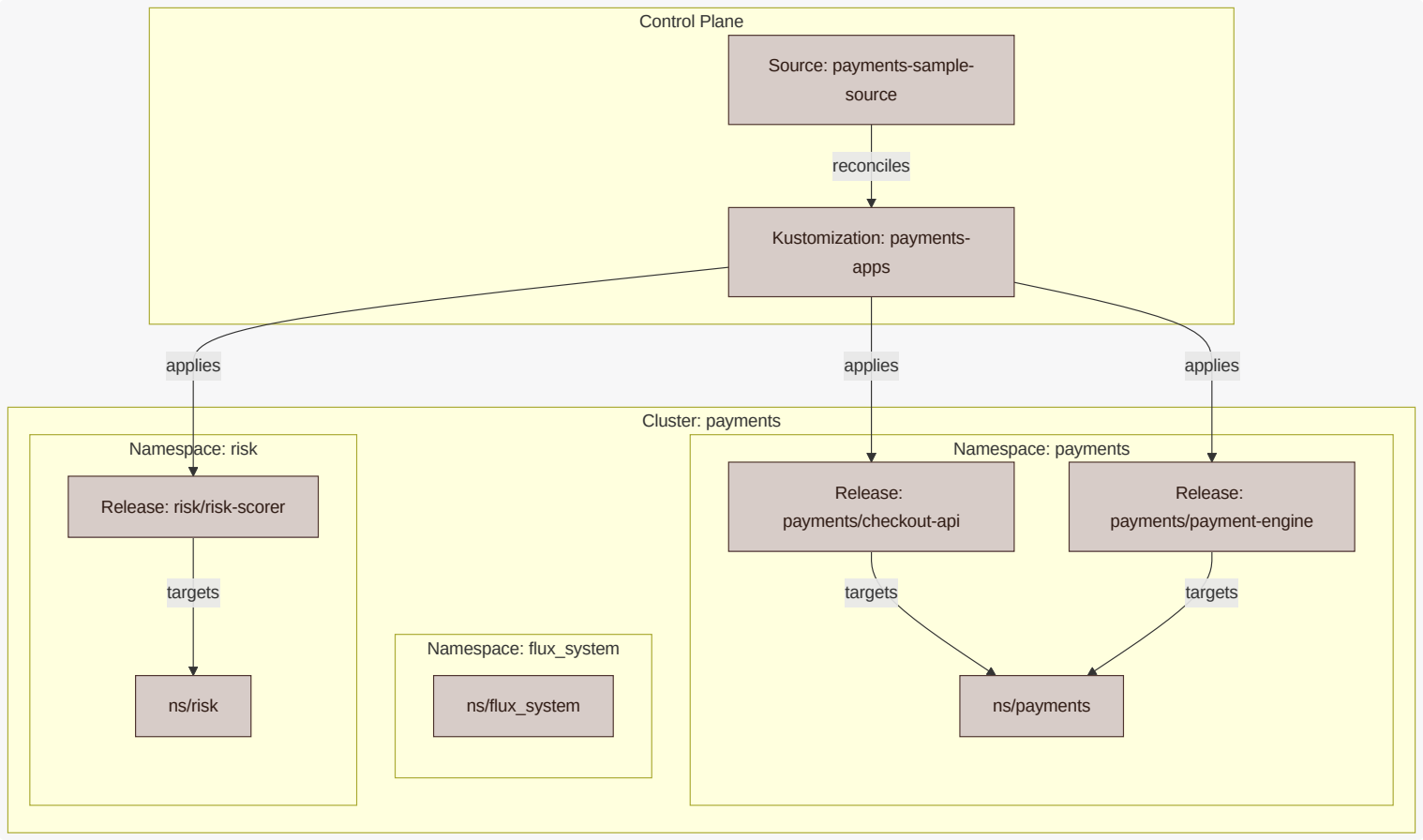
7.4. Recommended Next Evidence Additions

- Add owner metadata to Flux/Kustomization/`Helm` objects for clearer `platform` ownership mapping.
- Ensure deployment `control` artifacts (source, kustomization, release) are all included in scanned directories.
- Add deterministic naming conventions for release and namespace objects to improve cross-artifact linking.

7.5. Deployment Artifact Relationship Graph

What this diagram shows: how source/control-plane artifacts connect to releases, namespaces, workloads, and target cluster context.

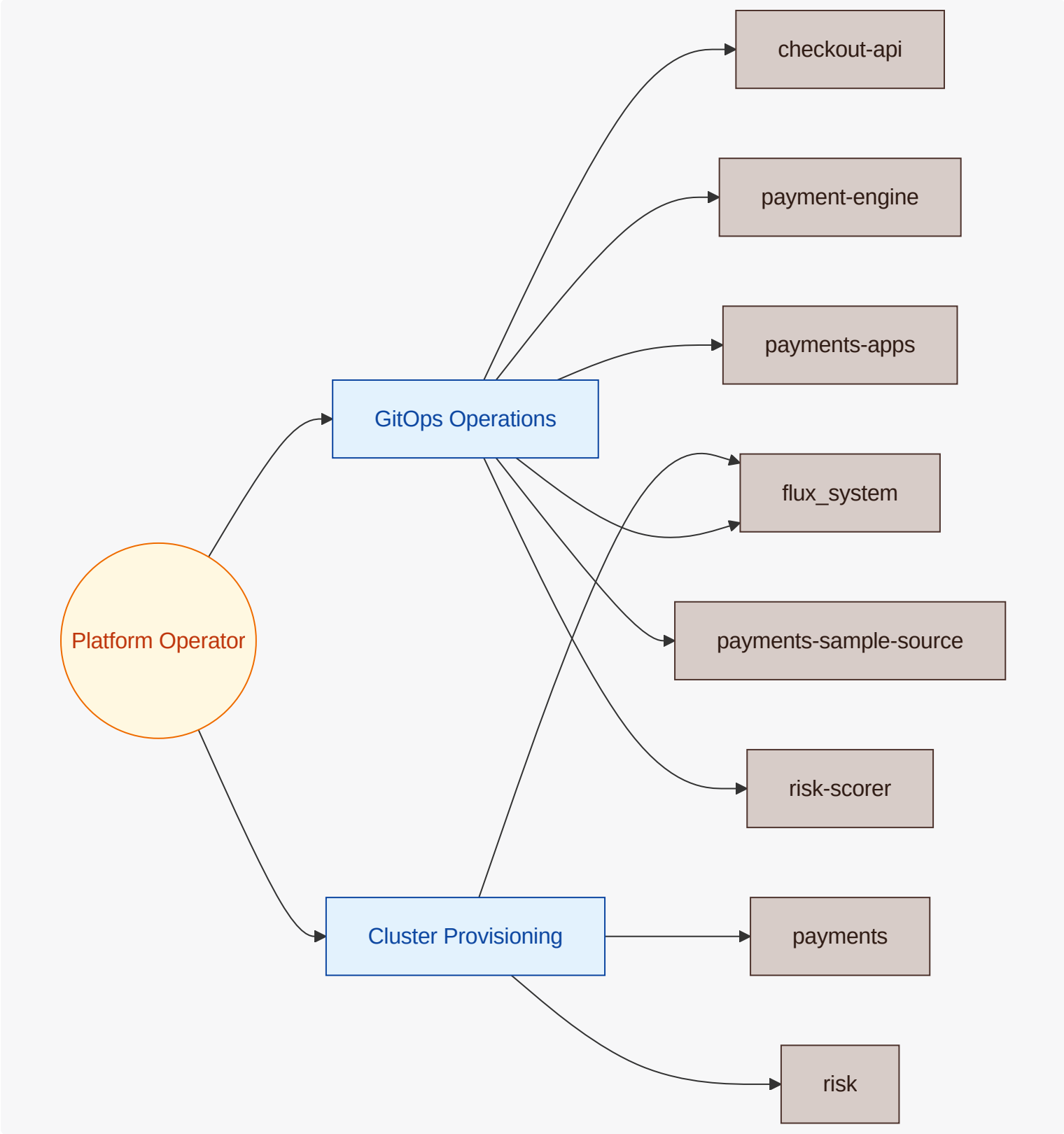
> **NOTE**
> This graph is inferred from available deployment artifacts (for example Terraform, Flux, Helm) and may be partial when source metadata is incomplete. Namespace and cluster boundaries are rendered as containers when target/placement relationships are inferred. Node labels include role hints (for example `Source`, `Kustomization`, `Release`, `Workload`, `ns/…`) to distinguish similarly named elements.



7.6. Platform Operations Graph

What this diagram shows: which platform actors operate which platform units and the control-plane targets they influence.

> **NOTE** Platform operations links are inferred from authored actor mappings plus discovered deployment control-plane artifacts.



7.7. Functional Groups and Units (View Scoped)

7.7.1. Payments (FG-PAYMENTS)

`payments` is deployed so `checkout` and authorization can roll independently with minimal blast radius. Deployment design prioritizes safe promotion, fast rollback, and dependency readiness checks before policy-impacting releases are exposed.

Checkout Handling (FU-CHECKOUT)

Checkout Handling Deployment Design

`checkout handling` is released independently to protect user-facing velocity without destabilizing policy logic. Deployment design favors fast rollback and compatibility checks with the current authorization `interface`.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-001`, `REQ-PAY-003`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-008`
Deployment dependencies	`Payment Authorization`
Inferred deployment evidence	runtime: `payments`, `payments/checkout-api`

Payment Authorization (FU-PAYMENT-AUTHORIZATION)

Payment Authorization Deployment Design

`payment authorization` deployment is policy-sensitive and therefore rollout-cautious. Releases should stage gradually, verify dependency health, and preserve in-flight transaction continuity.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-001`, `REQ-PAY-004`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-009`
Deployment dependencies	`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`
Inferred deployment evidence	runtime: `payments/payment-engine`

7.7.2. Fraud Evaluation (FG-FRAUD)

`fraud evaluation` deployment is tuned for cautious rollout because scoring changes can alter approval rates. Deployment design favors staged promotions and rapid monitoring of scoring outcomes before wider exposure.

Risk Scoring (FU-RISK-SCORING)

Risk Scoring Deployment Design

`risk scoring` deployment uses staged rollout because scoring changes have immediate decision impact. Deployment design includes observation windows for score distribution and escalation-rate changes.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-002`, `REQ-PAY-005`
Deployment dependencies	`PostgreSQL Client Driver`
Inferred deployment evidence	runtime: `risk`, `risk/risk-scorer`

Support Review (FU-SUPPORT-REVIEW)

Support Review Deployment Design

`support review` deployment isolates privileged workflows from public ingress components. Deployment design must preserve active review sessions across upgrades and ensure queue/review latency visibility.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	`REQ-PAY-004`, `REQ-PAY-009`
Deployment dependencies	none
Inferred deployment evidence	authored unit with no direct derived runtime/code evidence yet

7.7.3. Platform (FG-PLATFORM)

`platform` deployment design governs how infrastructure intent is applied and kept in sync over time. It prioritizes deterministic sequencing, drift `control`, and recoverable change paths before application teams depend on new `platform states`.

Cluster Provisioning (FU-CLUSTER-PROVISIONING)

Cluster Provisioning Deployment Design

`cluster provisioning` deployment is infrastructure-first and sequence-sensitive. Deployment design requires deterministic change plans, drift awareness, and rollback strategy before apply.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored `requirement` trigger
Deployment dependencies	none
Inferred deployment evidence	runtime: `payments`

GitOps Operations (FU-GITOPS-OPERATIONS)

GitOps Operations Deployment Design

`gitops operations` deployment defines the release `control` plane for application artifacts. Deployment design prioritizes policy gates, controlled promotion, and resilient rollback mechanics.

Deployment Aspect	View-Scoped Detail
Deployment trigger context	no explicit authored `requirement` trigger
Deployment dependencies	`Cluster Provisioning`, `Helm` Runtime `Platform`
Inferred deployment evidence	runtime: `flux-system/payments-apps`, `flux-system/payments-sample-source`, `flux_system`

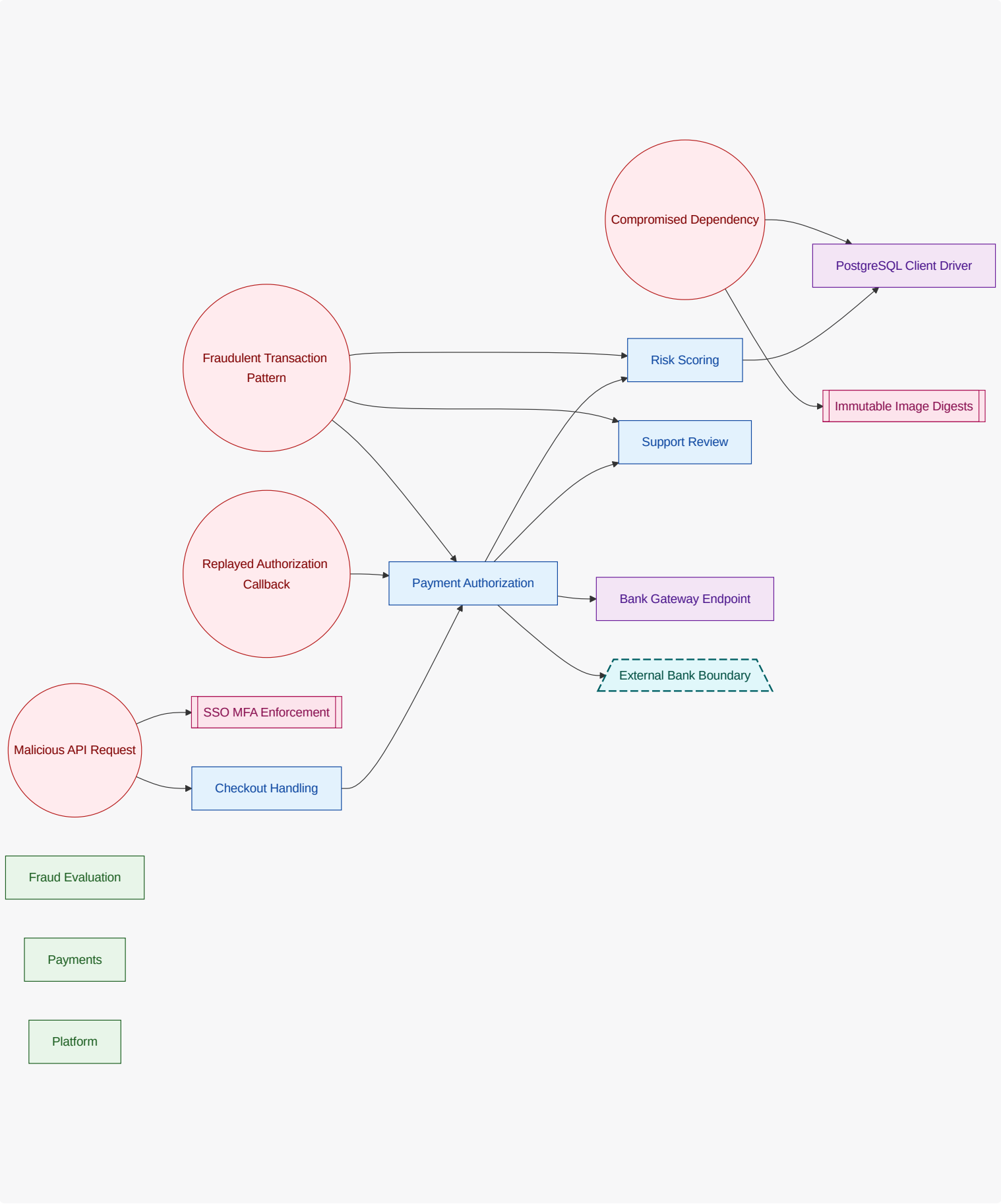
8. Security View

Show inferred exposure and dependency `risk` points aligned to unit boundaries, focused on attack paths and security evidence.

8.1. What This View Answers

- What are the main `trust boundaries` and attack paths?
- Which units are exposed and what security evidence exists?
- Where is the threat model incomplete?

8.2. View-Scoped Projection



Node ID	Kind	Label
AV-COMPROMISED-DEPENDENCY	attack_vector	Compromised Dependency
AV-FRAUDULENT-TRANSACTION-PATTERN	attack_vector	Fraudulent Transaction Pattern
AV-MALICIOUS-API-REQUEST	attack_vector	Malicious API Request
AV-REPLAYED-AUTH-CALLBACK	attack_vector	Replayed Authorization Callback
CTRL-PAYMENTS-IMAGE-DIGEST	control	Immutable Image Digests
CTRL-PAYMENTS-SSO-MFA	control	SSO MFA Enforcement
FG-FRAUD	functional_group	Fraud Evaluation
FG-PAYMENTS	functional_group	Payments
FG-PLATFORM	functional_group	Platform
FU-CHECKOUT	functional_unit	Checkout Handling
FU-PAYMENT-AUTHORIZATION	functional_unit	Payment Authorization
FU-RISK-SCORING	functional_unit	Risk Scoring
FU-SUPPORT-REVIEW	functional_unit	Support Review
REF-BANK-GATEWAY-ENDPOINT	referenced_element	Bank Gateway Endpoint
REF-POSTGRES-DRIVER	referenced_element	PostgreSQL Client Driver
TB-PAYMENTS-EXTERNAL-BANK	trust_boundary	External Bank Boundary

Mapping Type	From	To
bounded_by	FU-PAYMENT-AUTHORIZATION	TB-PAYMENTS-EXTERNAL-BANK

Mapping Type	From	To
depends_on	FU-CHECKOUT	FU-PAYMENT-AUTHORIZATION
depends_on	FU-PAYMENT-AUTHORIZATION	FU-RISK-SCORING
depends_on	FU-PAYMENT-AUTHORIZATION	FU-SUPPORT-REVIEW
depends_on	FU-PAYMENT-AUTHORIZATION	REF-BANK-GATEWAY-ENDPOINT
depends_on	FU-RISK-SCORING	REF-POSTGRES-DRIVER
mitigated_by	AV-COMPROMISED-DEPENDENCY	CTRL-PAYMENTS-IMAGE-DIGEST
mitigated_by	AV-MALICIOUS-API-REQUEST	CTRL-PAYMENTS-SSO-MFA
targets	AV-COMPROMISED-DEPENDENCY	REF-POSTGRES-DRIVER
targets	AV-FRAUDULENT-TRANSACTION-PATTERN	FU-PAYMENT-AUTHORIZATION
targets	AV-FRAUDULENT-TRANSACTION-PATTERN	FU-RISK-SCORING
targets	AV-FRAUDULENT-TRANSACTION-PATTERN	FU-SUPPORT-REVIEW
targets	AV-MALICIOUS-API-REQUEST	FU-CHECKOUT
targets	AV-REPLAYED-AUTH-CALLBACK	FU-PAYMENT-AUTHORIZATION

8.3. Coverage Gaps

Coverage summary: 4/4 units have direct evidence coverage in this view.

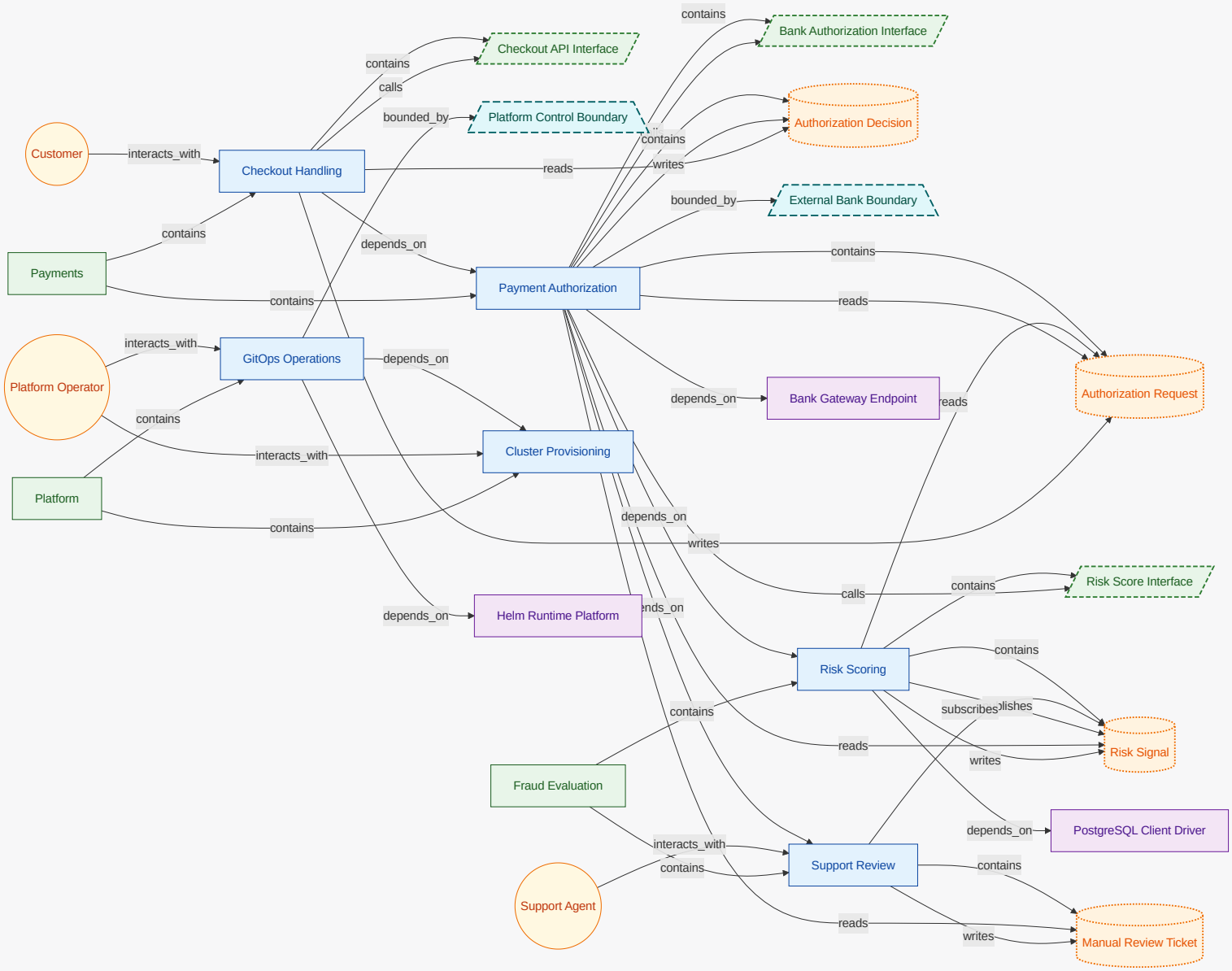
- No major coverage gaps detected from current authored and inferred inputs.

8.4. Recommended Next Evidence Additions

- No immediate evidence additions are required for this view.

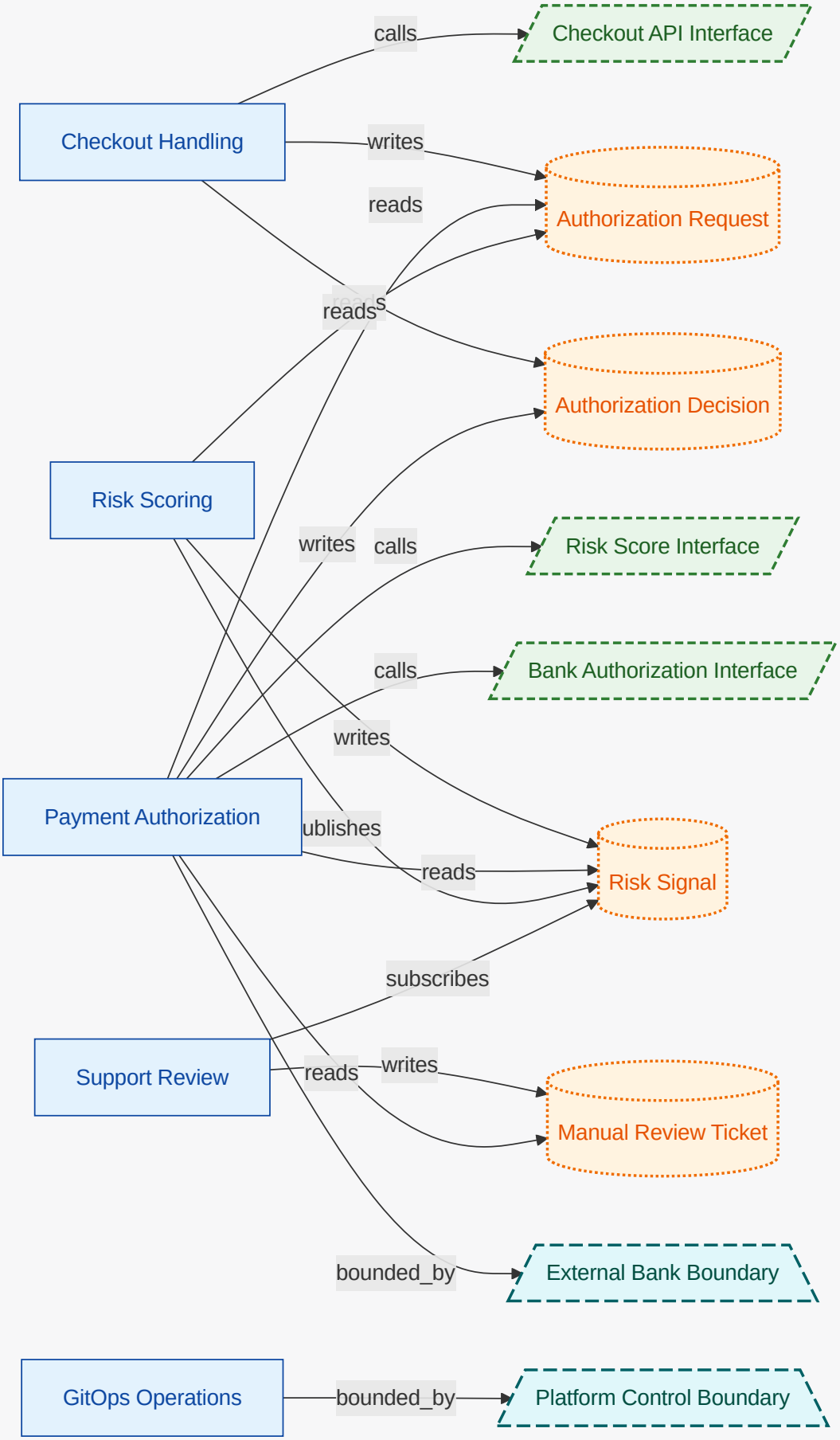
8.5. Threat Model Context Diagram

What this diagram shows: OWASP-style context view of actors, core processes, interfaces, and trust boundary-relevant entities.


Customer
interacts_with
Checkout Handling
contains
calls
Checkout API Interface
bounded_by
Platform Control Boundary
reads
contains
Payments
contains
contains
depends_on
Payment Authorization
contains
Bank Authorization Interface
contains
writes
Authorization Decision
bounded_by
External Bank Boundary
contains
reads
Authorization Request
Platform Operator
interacts_with
GitOps Operations
depends_on
contains
interacts_with
Cluster Provisioning
Platform
contains
depends_on
Bank Gateway Endpoint
reads
depends_on
writes
depends_on
Helm Runtime Platform
calls
contains
Risk Score Interface
Risk Scoring
contains
subscribes
lishes
reads
writes
Risk Signal
Fraud Evaluation
contains
depends_on
PostgreSQL Client Driver
interacts_with
contains
Support Review
contains
Support Agent
reads
writes
Manual Review Ticket

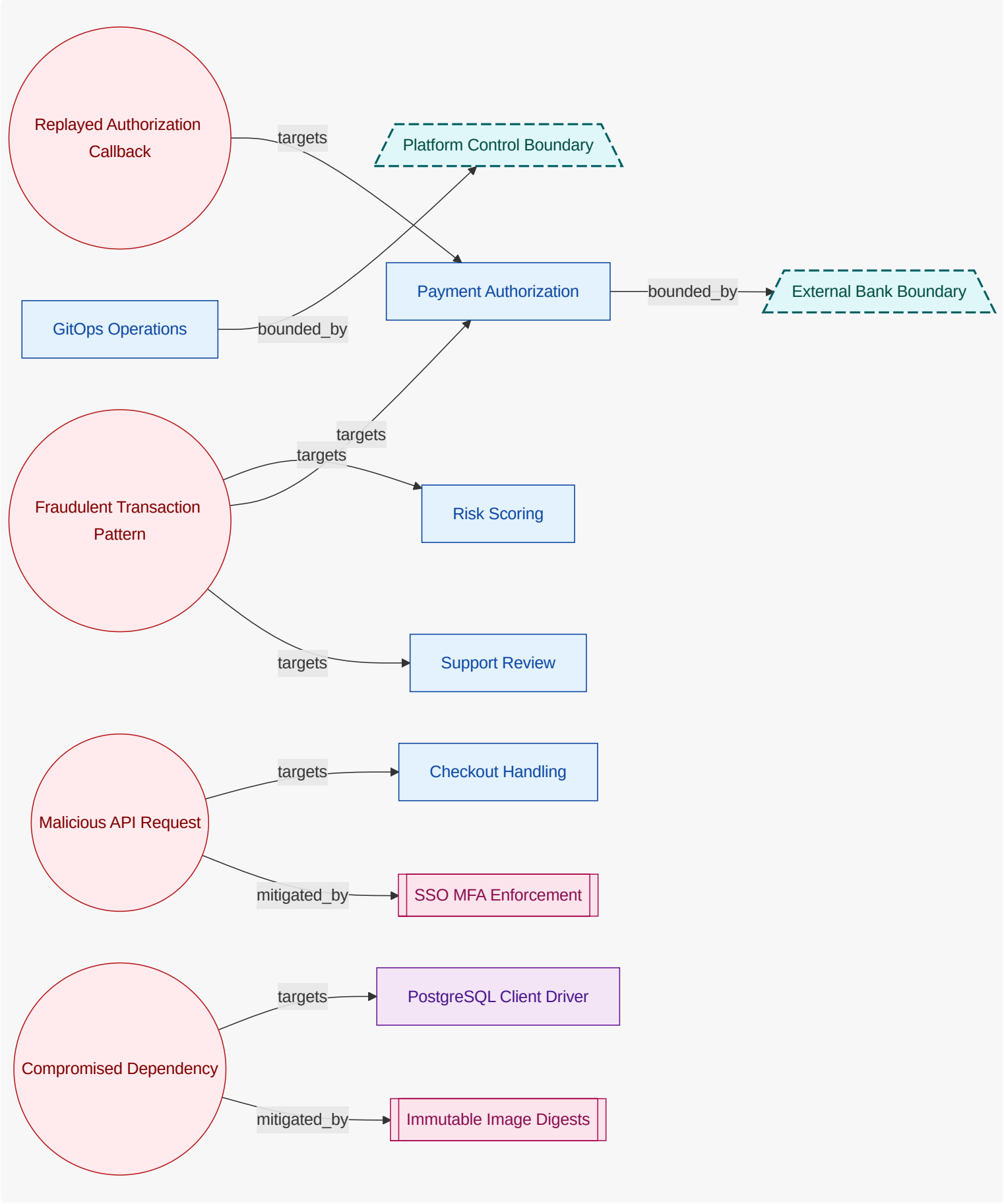
8.6. Threat Model Data Flow Diagram

What this diagram shows: primary security-relevant data flows (reads/writes/publishes/subscribes/streams) between processes and data objects.


Checkout API Interface
calls
Checkout Handling
writes
Authorization Request
reads
reads
Authorization Decision
Risk Scoring
writes
calls
Risk Score Interface
calls
Bank Authorization Interface
writes
Payment Authorization
ublishes
reads
Risk Signal
subscribes
Support Review
reads
writes
Manual Review Ticket
bounded_by
External Bank Boundary
GitOps Operations
bounded_by
Platform Control Boundary

8.7. Threat-to-Control Overlay Diagram

What this diagram shows: attack vectors, targeted components, and mitigation/boundary relationships.


Replayed Authorization Callback
targets
Platform Control Boundary
Payment Authorization
bounded_by
External Bank Boundary
GitOps Operations
bounded_by
targets
targets
Fraudulent Transaction Pattern
Risk Scoring
targets
Support Review
Malicious API Request
targets
Checkout Handling
mitigated_by
SSO MFA Enforcement
Compromised Dependency
targets
PostgreSQL Client Driver
mitigated_by
Immutable Image Digests

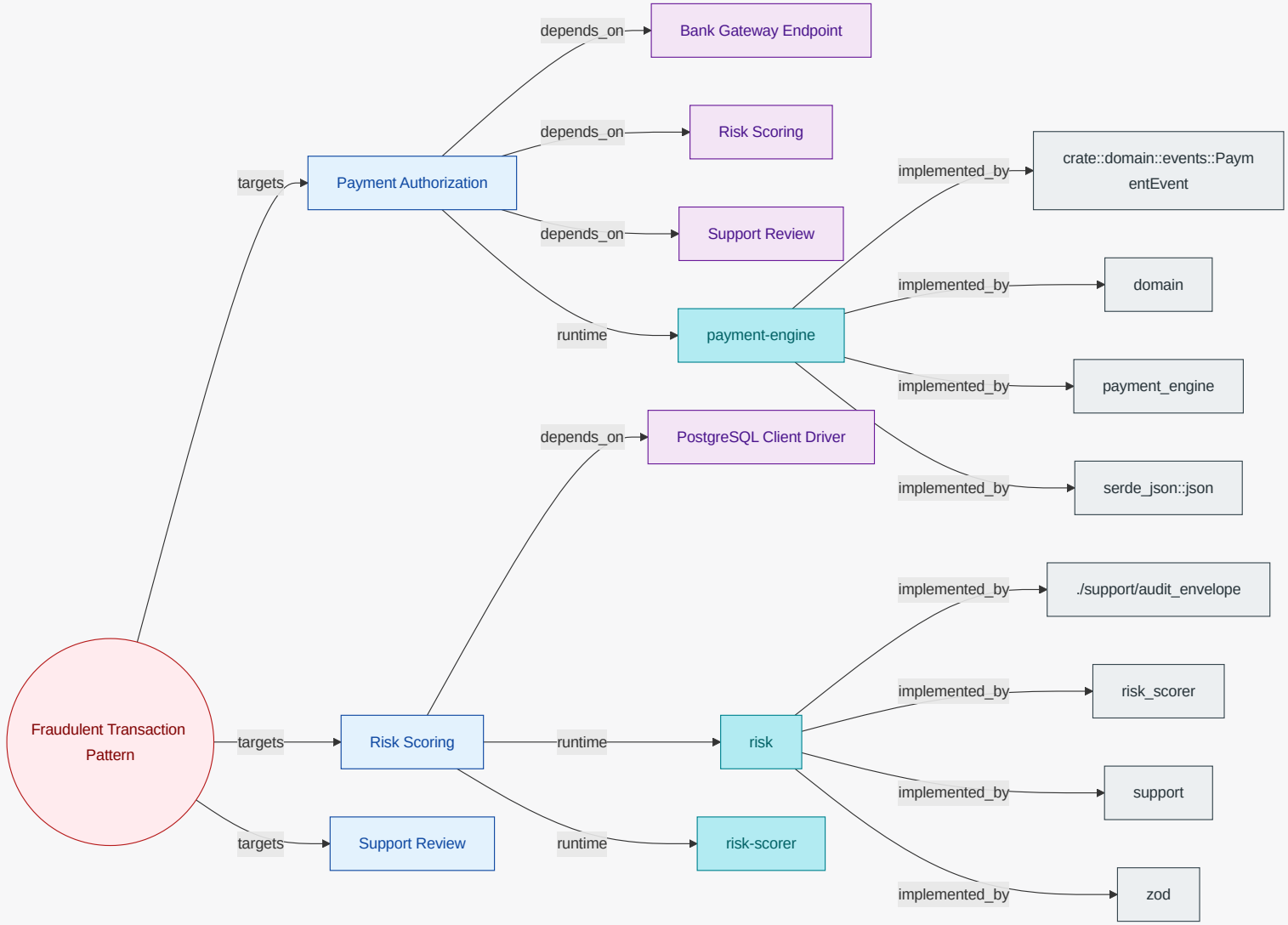
8.8. Security Logging and Observability

Signal	Layer	Owner	Evidence
application security telemetry hooks	code	FU-RISK-SCORING	code support/audit_envelope.ts
fraud and abuse detection code paths	code	FU-RISK-SCORING	code risk_scorer.ts
deployment and platform audit events	deployment	FU-CHECKOUT	helmrelease checkout-api
deployment and platform audit events	deployment	FU-CHECKOUT	namespace payments
deployment and platform audit events	deployment	FU-CLUSTER-PROVISIONING	cluster payments
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	gitrepository payments-sample-source
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	kustomization payments-apps
deployment and platform audit events	deployment	FU-GITOPS-OPERATIONS	namespace flux_system
deployment and platform audit events	deployment	FU-PAYMENT-AUTHORIZATION	helmrelease payment-engine
deployment and platform audit events	deployment	FU-RISK-SCORING	helmrelease risk-scorer
deployment and platform audit events	deployment	FU-RISK-SCORING	namespace risk
abuse and fraud decision logs	runtime	FU-RISK-SCORING	helmrelease risk-scorer
abuse and fraud decision logs	runtime	FU-RISK-SCORING	namespace risk

8.9. Attack Vector Chapters

8.9.1. `Fraudulent Transaction Pattern` (AV-FRAUDULENT-TRANSACTION-PATTERN)

Coordinated abuse pattern using synthetic identities, stolen instruments, and velocity anomalies to seek unauthorized approvals.



Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`payment authorization` security protects decision integrity and callback integrity. The design defends against replay, policy probing, and ambiguous error channels that could be abused.

Attack Surface

`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`

Security Evidence Present

runtime: `payments/payment-engine` | code modules: domain, `payment_engine`

Risk Scoring (FU-RISK-SCORING)

`risk scoring` security assumes adversarial input attempts and model manipulation pressure. Design controls enforce strict input handling and evidence capture for post-event investigation.

Attack Surface

`PostgreSQL Client Driver`

Security Evidence Present

runtime: `risk`, `risk/risk-scorer` | code modules: `risk_scorer`, `support`

Support Review (FU-SUPPORT-REVIEW)

`support review` security requires strong operator identity controls and tamper-evident audit trails. The design prevents silent overrides and guarantees every manual decision is attributable.

Attack Surface

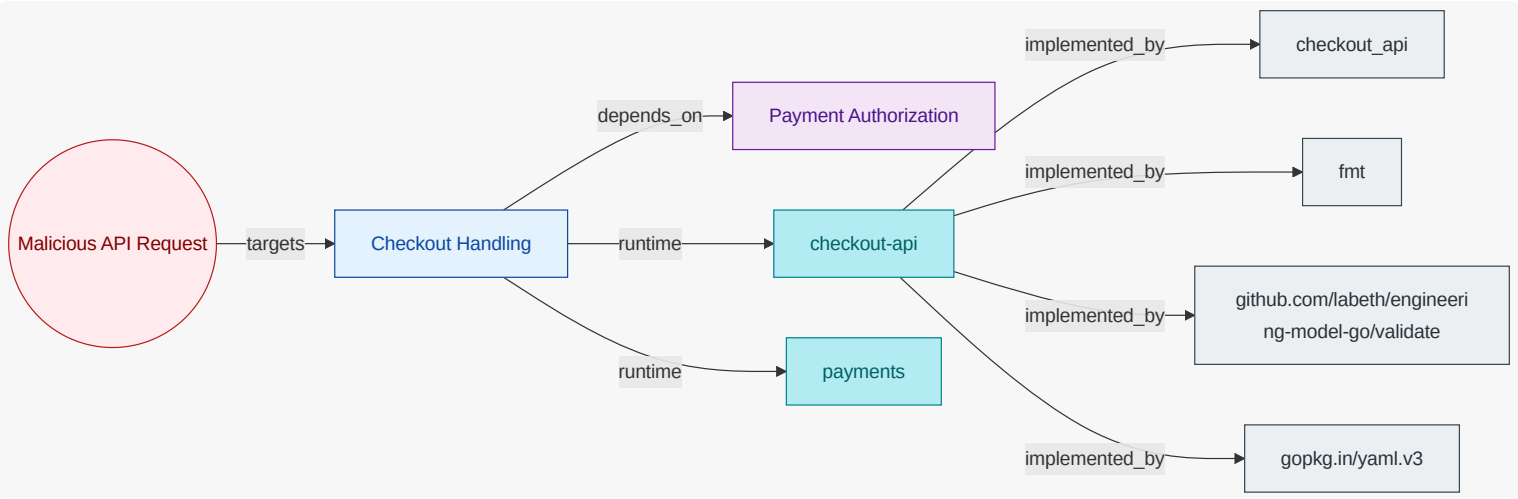
none

Security Evidence Present

authored unit with no direct derived runtime/code evidence yet

8.9.2. `Malicious API Request` (AV-MALICIOUS-API-REQUEST)

Crafted payloads attempting to bypass authorization controls.



Checkout Handling (FU-CHECKOUT)

`checkout handling` security protects the internet-facing ingress. The design requires strict validation, abuse resistance, and traceable rejection behavior so malicious inputs are contained without harming legitimate `customer` flows.

Attack Surface

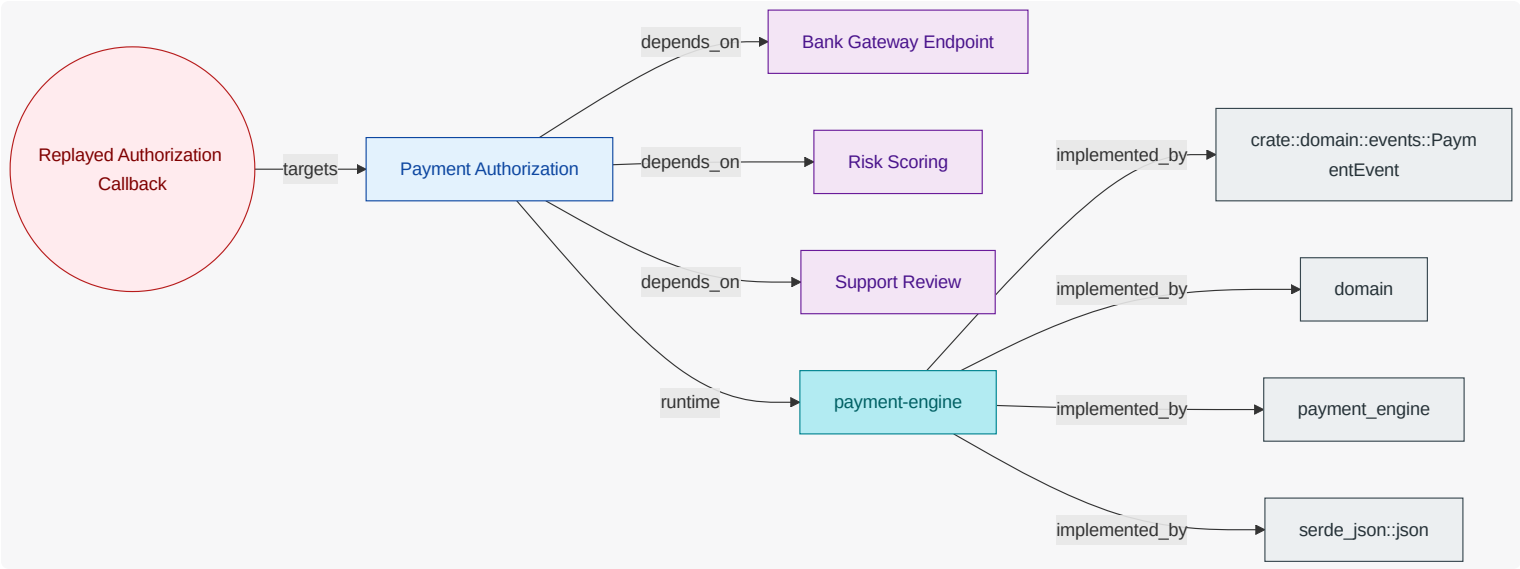
`Payment Authorization`

Security Evidence Present

runtime: `payments`, `payments/checkout-api` | code modules: `checkout_api`

8.9.3. `Replayed Authorization Callback` (AV-REPLAYED-AUTH-CALLBACK)

Replay attack against external authorization callback `flow`.



Payment Authorization (FU-PAYMENT-AUTHORIZATION)

`payment authorization` security protects decision integrity and callback integrity. The design defends against replay, policy probing, and ambiguous error channels that could be abused.

Attack Surface

`Bank Gateway Endpoint`, `Risk Scoring`, `Support Review`

Security Evidence Present

runtime: `payments/payment-engine` | code modules: domain, `payment_engine`

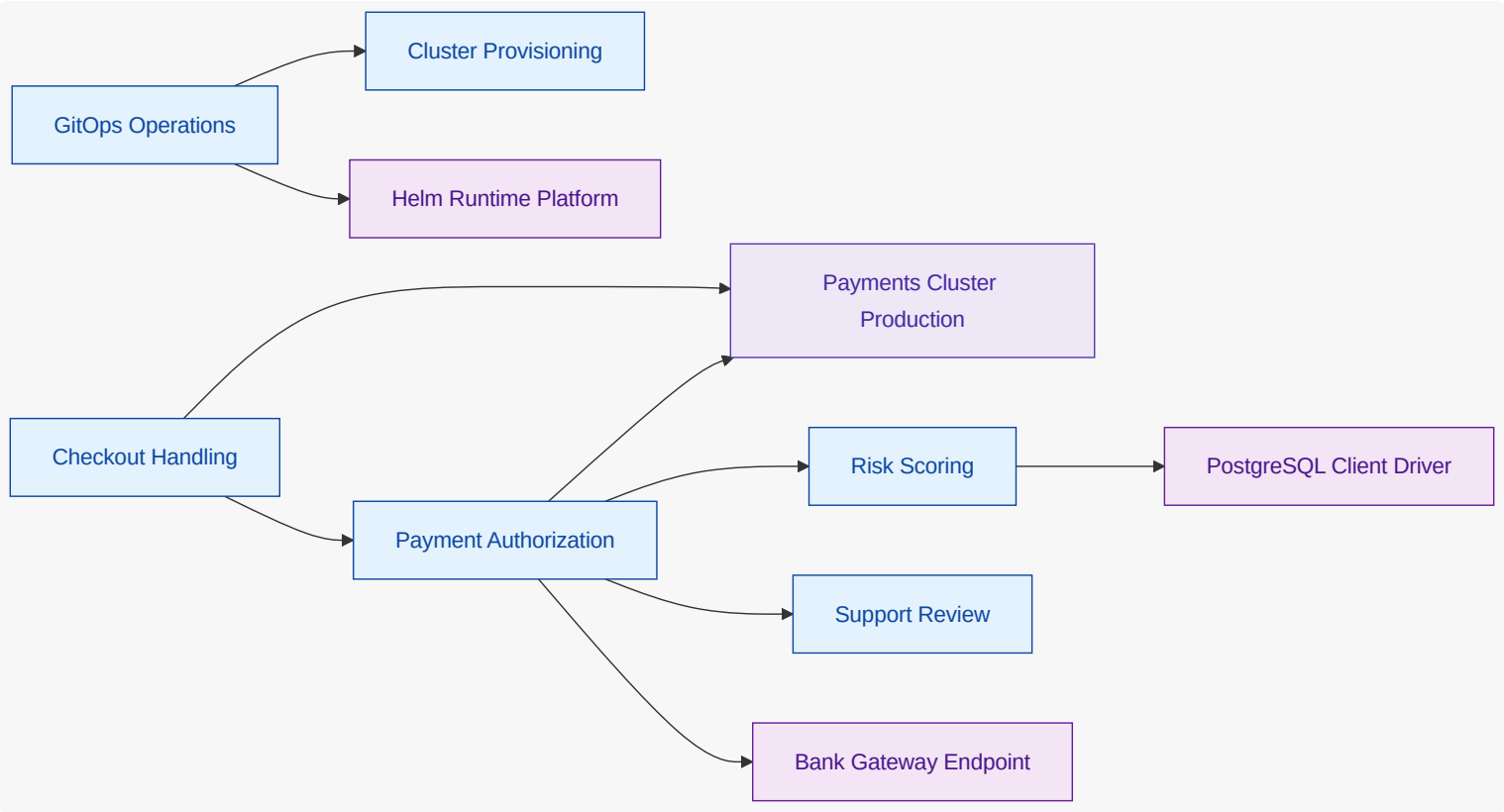
9. Traceability View

Show requirement-to-unit-to-evidence traceability, including coverage confidence and explicit evidence gaps.

9.1. What This View Answers

- Which `requirements` map to which units?
- Which authored units have runtime/code evidence and which do not?
- Where are confidence or completeness gaps in trace coverage?

9.2. View-Scoped Projection



Node ID	Kind	Label
`DEP-PAYMENTS-CLUSTER-PROD`	deployment_target	`Payments Cluster Production`
`FU-CHECKOUT`	functional_unit	`Checkout Handling`
`FU-CLUSTER-PROVISIONING`	functional_unit	`Cluster Provisioning`
`FU-GITOPS-OPERATIONS`	functional_unit	`GitOps Operations`

Node ID	Kind	Label
`FU-PAYMENT-AUTHORIZATION`	functional_unit	`Payment Authorization`
`FU-RISK-SCORING`	functional_unit	`Risk Scoring`
`FU-SUPPORT-REVIEW`	functional_unit	`Support Review`
`REF-BANK-GATEWAY-ENDPOINT`	referenced_element	`Bank Gateway Endpoint`
`REF-HELM-PLATFORM`	referenced_element	`Helm Runtime Platform`
`REF-POSTGRES-DRIVER`	referenced_element	`PostgreSQL Client Driver`

Mapping Type	From	To
depends_on	`FU-CHECKOUT`	`FU-PAYMENT-AUTHORIZATION`
depends_on	`FU-GITOPS-OPERATIONS`	`FU-CLUSTER-PROVISIONING`
depends_on	`FU-GITOPS-OPERATIONS`	`REF-HELM-PLATFORM`
depends_on	`FU-PAYMENT-AUTHORIZATION`	`FU-RISK-SCORING`
depends_on	`FU-PAYMENT-AUTHORIZATION`	`FU-SUPPORT-REVIEW`
depends_on	`FU-PAYMENT-AUTHORIZATION`	`REF-BANK-GATEWAY-ENDPOINT`
depends_on	`FU-RISK-SCORING`	`REF-POSTGRES-DRIVER`
deployed_to	`FU-CHECKOUT`	`DEP-PAYMENTS-CLUSTER-PROD`
deployed_to	`FU-PAYMENT-AUTHORIZATION`	`DEP-PAYMENTS-CLUSTER-PROD`

9.3. Coverage Gaps

Coverage summary: 5/6 units have direct evidence coverage in this view.

- `Cluster Provisioning` has no direct inferred code evidence yet.
- `GitOps Operations` has no direct inferred code evidence yet.
- `Support Review` has no direct inferred code evidence yet.
- `Support Review` has no direct inferred runtime evidence yet.

9.4. Recommended Next Evidence Additions

- Add explicit requirement-to-owner mappings where `requirement` scope is currently broad or ambiguous.
- Add ownership annotations for runtime and code artifacts where coverage is still unresolved.
- Prioritize closing gaps for units that are authored but lack both runtime and code evidence.

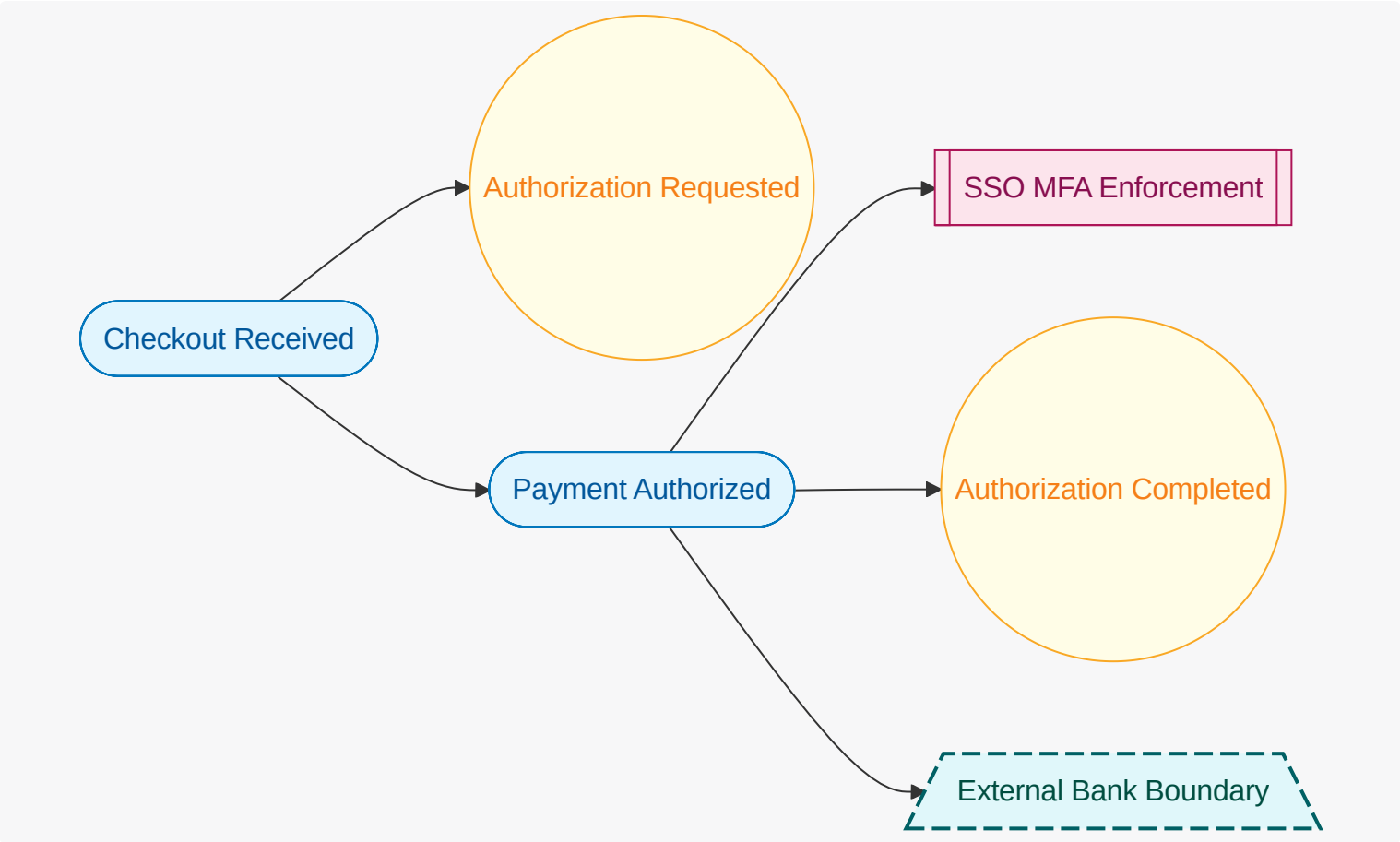
10. State Lifecycle View

Show workflow `state` transitions, ownership of transitions, and key exceptional paths.

10.1. What This View Answers

- What `states` exist and which `events` trigger transitions?
- Which unit owns each `state` transition?
- Where do automatic and manual flows diverge?

10.2. View-Scoped Projection



Node ID	Kind	Label
`CTRL-PAYMENTS-SSO-MFA`	`control`	`SSO MFA Enforcement`
`EVT-PAYMENTS-AUTH-COMPLETED`	`event`	`Authorization Completed`
`EVT-PAYMENTS-AUTH-REQUESTED`	`event`	`Authorization Requested`

Node ID	Kind	Label
STATE-PAYMENTS-AUTHORIZED	state	Payment Authorized
STATE-PAYMENTS-CHECKOUT-RECEIVED	state	Checkout Received
TB-PAYMENTS-EXTERNAL-BANK	trust_boundary	External Bank Boundary

Mapping Type	From	To
guarded_by	STATE-PAYMENTS-AUTHORIZED	CTRL-PAYMENTS-SSO-MFA
guarded_by	STATE-PAYMENTS-AUTHORIZED	TB-PAYMENTS-EXTERNAL-BANK
transitions_to	STATE-PAYMENTS-CHECKOUT-RECEIVED	STATE-PAYMENTS-AUTHORIZED
triggered_by	STATE-PAYMENTS-AUTHORIZED	EVT-PAYMENTS-AUTH-COMPLETED
triggered_by	STATE-PAYMENTS-CHECKOUT-RECEIVED	EVT-PAYMENTS-AUTH-REQUESTED

10.3. Coverage Gaps

Coverage summary: No functional units are in scope for this view.

- No major coverage gaps detected from current authored and inferred inputs.

10.4. Recommended Next Evidence Additions

- No immediate evidence additions are required for this view.

10.5. Functional Groups and Units (View Scoped)

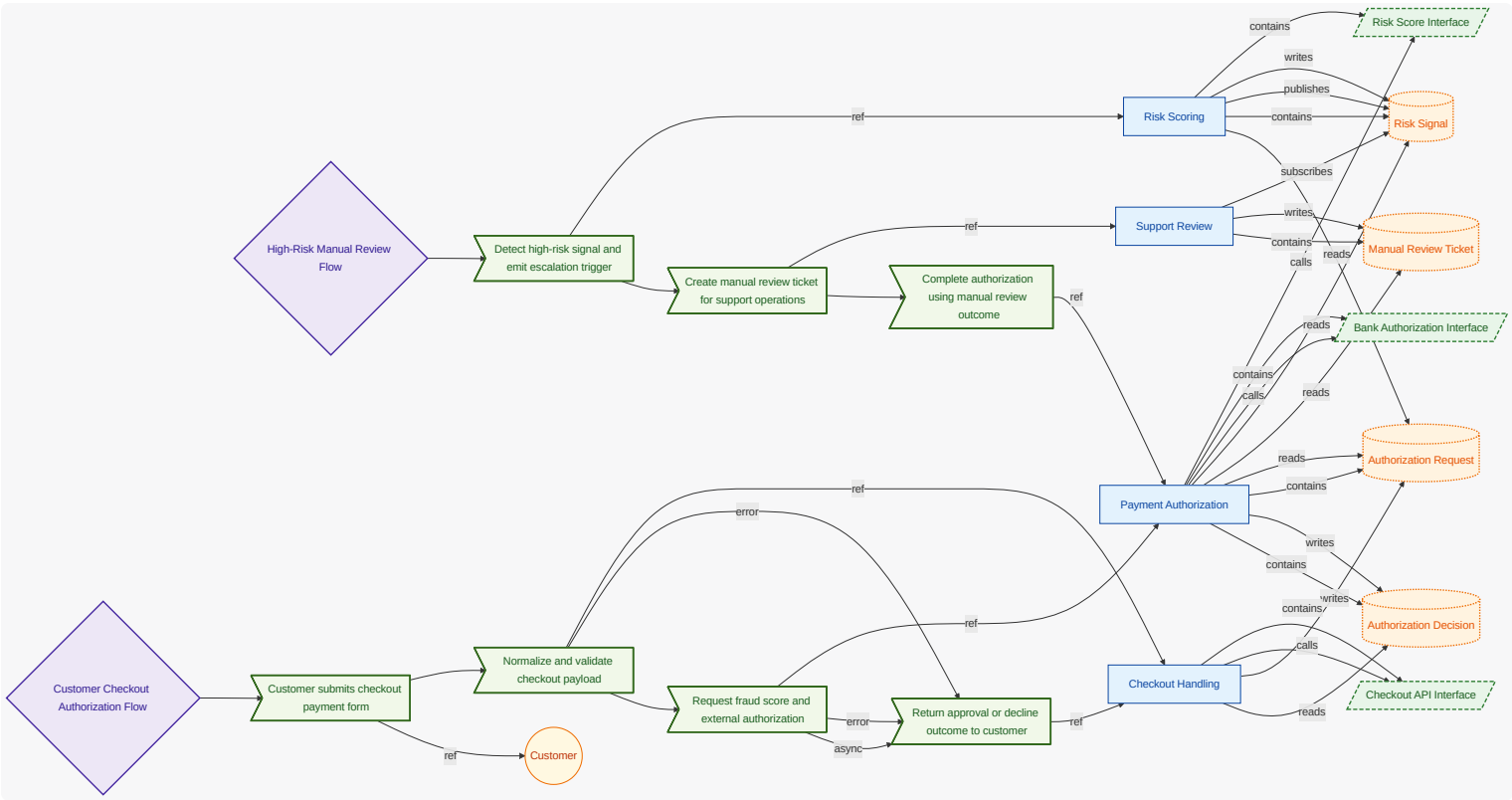
11. Interaction Flow View

Show user-intent-to-outcome causal `flow`, including ordered steps, data movement, and error/async branches.

11.1. What This View Answers

- What happens from user intent to system outcome, step by step?
- What data enters/exits each step and where are decisions made?
- Where are async and error paths and what completes each path?

11.2. View-Scoped Projection



Node ID	Kind	Label
ACT-CUSTOMER	actor	Customer
DO-PAYMENTS-AUTH-DECISION	data_object	Authorization Decision
DO-PAYMENTS-AUTH-REQUEST	data_object	Authorization Request
DO-PAYMENTS-REVIEW-TICKET	data_object	Manual Review Ticket

Node ID	Kind	Label
`DO-PAYMENTS-RISK-SIGNAL`	data_object	`Risk Signal`
`FLOW-CUSTOMER-CHECKOUT`	`flow`	`Customer Checkout Authorization Flow`
`FLOW-PAYMENTS-MANUAL-REVIEW`	`flow`	`High-Risk Manual Review Flow`
`FLOW-CUSTOMER-CHECKOUT::authorize-payment`	flow_step	`Request fraud score and external authorization`
`FLOW-CUSTOMER-CHECKOUT::normalize-request`	flow_step	`Normalize and validate checkout payload`
`FLOW-CUSTOMER-CHECKOUT::show-outcome`	flow_step	`Return approval or decline outcome to customer`
`FLOW-CUSTOMER-CHECKOUT::submit-payment`	flow_step	`Customer submits checkout payment form`
`FLOW-PAYMENTS-MANUAL-REVIEW::complete-review`	flow_step	`Complete authorization using manual review outcome`
`FLOW-PAYMENTS-MANUAL-REVIEW::create-review-ticket`	flow_step	`Create manual review ticket for support operations`
`FLOW-PAYMENTS-MANUAL-REVIEW::detect-high-risk`	flow_step	`Detect high-risk signal and emit escalation trigger`
`FU-CHECKOUT`	functional_unit	`Checkout Handling`
`FU-PAYMENT-AUTHORIZATION`	functional_unit	`Payment Authorization`
`FU-RISK-SCORING`	functional_unit	`Risk Scoring`

Node ID	Kind	Label
FU-SUPPORT-REVIEW	functional_unit	Support Review
IF-PAYMENTS-BANK-AUTH	interface	Bank Authorization Interface
IF-PAYMENTS-CHECKOUT-API	interface	Checkout API Interface
IF-PAYMENTS-RISK-SCORE-API	interface	Risk Score Interface

Mapping Type	From	To
calls	FU-CHECKOUT	IF-PAYMENTS-CHECKOUT-API
calls	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-BANK-AUTH
calls	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-RISK-SCORE-API
contains	FU-CHECKOUT	IF-PAYMENTS-CHECKOUT-API
contains	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-DECISION
contains	FU-PAYMENT-AUTHORIZATION	DO-PAYMENTS-AUTH-REQUEST
contains	FU-PAYMENT-AUTHORIZATION	IF-PAYMENTS-BANK-AUTH
contains	FU-RISK-SCORING	DO-PAYMENTS-RISK-SIGNAL
contains	FU-RISK-SCORING	IF-PAYMENTS-RISK-SCORE-API
contains	FU-SUPPORT-REVIEW	DO-PAYMENTS-REVIEW-TICKET
flow_async	FLOW-CUSTOMER-CHECKOUT::authorize-payment	FLOW-CUSTOMER-CHECKOUT::show-outcome
flow_error	FLOW-CUSTOMER-CHECKOUT::authorize-payment	FLOW-CUSTOMER-CHECKOUT::show-outcome
flow_error	FLOW-CUSTOMER-CHECKOUT::normalize-request	FLOW-CUSTOMER-CHECKOUT::show-outcome
flow_next	FLOW-CUSTOMER-CHECKOUT	FLOW-CUSTOMER-CHECKOUT::submit-payment
flow_next	FLOW-CUSTOMER-CHECKOUT::normalize-request	FLOW-CUSTOMER-CHECKOUT::authorize-payment

Mapping Type	From	To
flow_next	`FLOW-CUSTOMER-CHECKOUT::submit-payment`	`FLOW-CUSTOMER-CHECKOUT::normalize-request`
flow_next	`FLOW-PAYMENTS-MANUAL-REVIEW`	`FLOW-PAYMENTS-MANUAL-REVIEW::detect-high-risk`
flow_next	`FLOW-PAYMENTS-MANUAL-REVIEW::create-review-ticket`	`FLOW-PAYMENTS-MANUAL-REVIEW::complete-review`
flow_next	`FLOW-PAYMENTS-MANUAL-REVIEW::detect-high-risk`	`FLOW-PAYMENTS-MANUAL-REVIEW::create-review-ticket`
flow_ref	`FLOW-CUSTOMER-CHECKOUT::authorize-payment`	`FU-PAYMENT-AUTHORIZATION`
flow_ref	`FLOW-CUSTOMER-CHECKOUT::normalize-request`	`FU-CHECKOUT`
flow_ref	`FLOW-CUSTOMER-CHECKOUT::show-outcome`	`FU-CHECKOUT`
flow_ref	`FLOW-CUSTOMER-CHECKOUT::submit-payment`	`ACT-CUSTOMER`
flow_ref	`FLOW-PAYMENTS-MANUAL-REVIEW::complete-review`	`FU-PAYMENT-AUTHORIZATION`
flow_ref	`FLOW-PAYMENTS-MANUAL-REVIEW::create-review-ticket`	`FU-SUPPORT-REVIEW`
flow_ref	`FLOW-PAYMENTS-MANUAL-REVIEW::detect-high-risk`	`FU-RISK-SCORING`
publishes	`FU-RISK-SCORING`	`DO-PAYMENTS-RISK-SIGNAL`
reads	`FU-CHECKOUT`	`DO-PAYMENTS-AUTH-DECISION`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-AUTH-REQUEST`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-REVIEW-TICKET`
reads	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-RISK-SIGNAL`
reads	`FU-RISK-SCORING`	`DO-PAYMENTS-AUTH-REQUEST`
subscribes	`FU-SUPPORT-REVIEW`	`DO-PAYMENTS-RISK-SIGNAL`
writes	`FU-CHECKOUT`	`DO-PAYMENTS-AUTH-REQUEST`
writes	`FU-PAYMENT-AUTHORIZATION`	`DO-PAYMENTS-AUTH-DECISION`

Mapping Type	From	To
writes	`FU-RISK-SCORING`	`DO-PAYMENTS-RISK-SIGNAL`
writes	`FU-SUPPORT-REVIEW`	`DO-PAYMENTS-REVIEW-TICKET`

11.3. Coverage Gaps

Coverage summary: 4/4 units have direct evidence coverage in this view.

- No major coverage gaps detected from current authored and inferred inputs.

11.4. Recommended Next Evidence Additions

- No immediate evidence additions are required for this view.

11.5. Functional Groups and Units (View Scoped)

11.6. Requirement-to-Unit Mapping (Compact)

What this matrix shows: authored requirement-to-unit mappings in a compact square layout (requirements as rows, functional units as columns).

Requirement	FU-CHECKOUT	FU-PAYMENT-AUTHORIZATION	FU-RISK-SCORING	FU-SUPPORT-REVIEW
REQ-PAY-001	X	X		
REQ-PAY-002			X	
REQ-PAY-003	X			
REQ-PAY-004		X		X
REQ-PAY-005			X	
REQ-PAY-006	X	X		
REQ-PAY-007	X	X		
REQ-PAY-008	X			
REQ-PAY-009		X		X

11.7. Verification Result Mapping

What this section shows: per-requirement result mapping inferred from test-result artifacts.

Check ID	Requirement	Result	Evidence / Notes
`VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597`	`REQ-PAY-001`	not-run	none No parsed test-result artifact for this `requirement` yet.
`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`	`REQ-PAY-002`	pass	test-results/integration-fraud-gate.json
`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`	`REQ-PAY-003`	pass	test-results/contract-checkout-decline.json
`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`	`REQ-PAY-004`	pass	test-results/integration-fraud-gate.json
`VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2`	`REQ-PAY-005`	partial	test-results/audit-persistence-report.md
`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`	`REQ-PAY-006`	pass	test-results/contract-checkout-decline.json

12. Traceability Appendix

12.1. Requirement Details

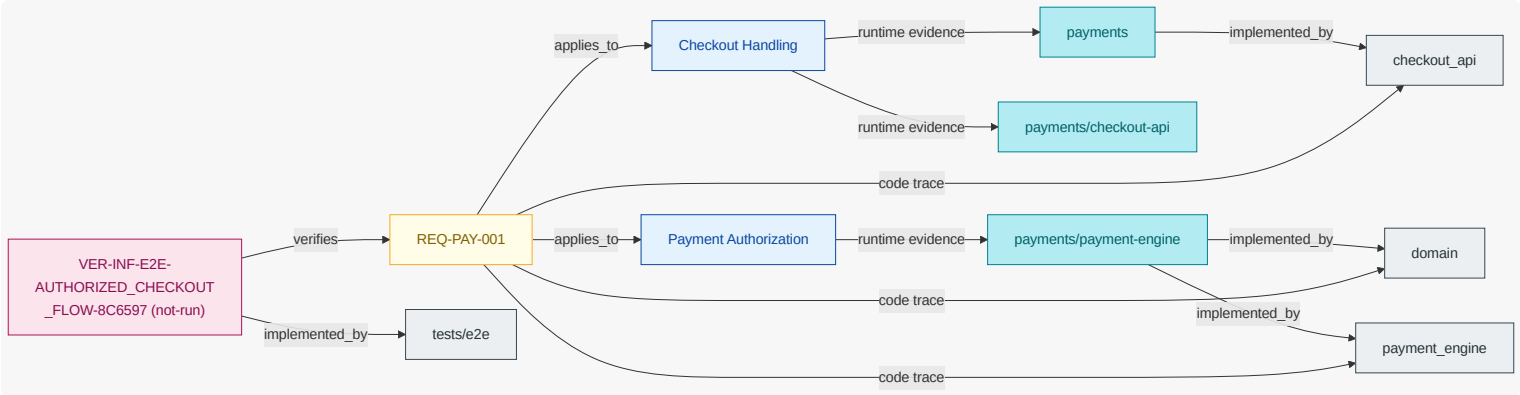
12.1.1. REQ-PAY-001

While `normal mode is active`, when `payment authorization is requested`, `the payments system` shall create a payment session.

> **NOTE** Core authorization path from `checkout` to bank gateway.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



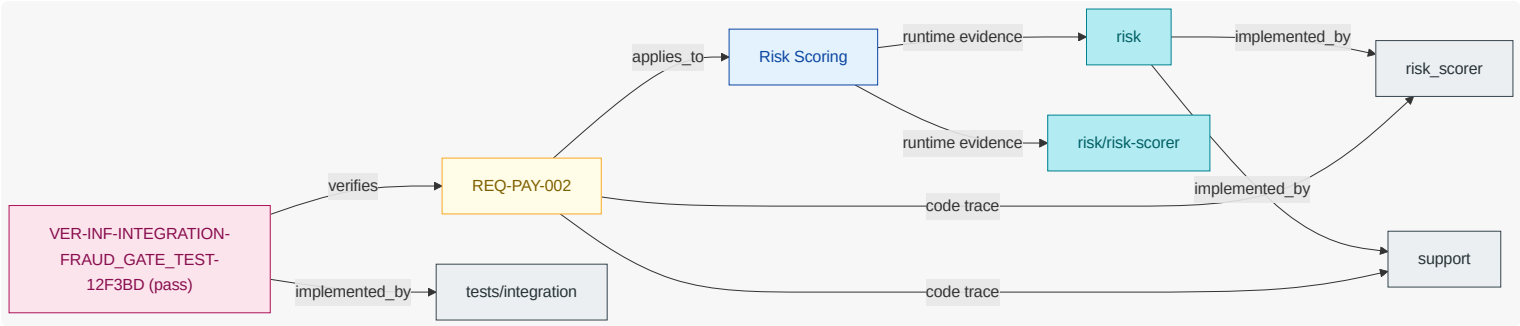
12.1.2. REQ-PAY-002

Where `fraud check is enabled`, when `payment authorization is requested`, `the fraud detection system` shall calculate a `risk score` before approval.

> **NOTE** `Fraud scoring` gate before approval.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



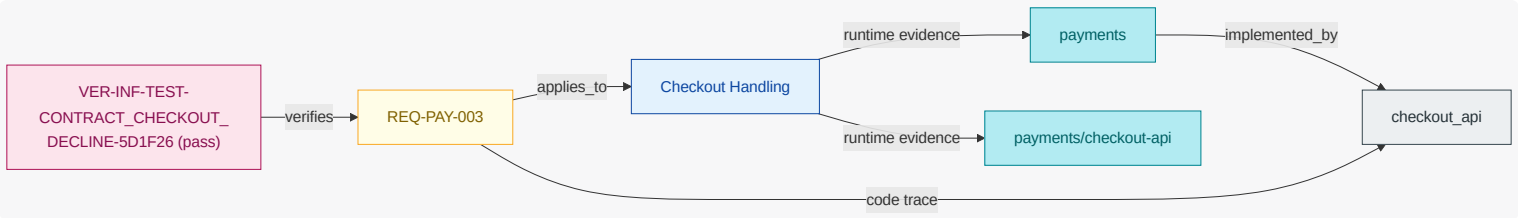
12.1.3. REQ-PAY-003

If `payment is declined`, then `the payments system` shall present a `decline reason` to `customer`.

> **NOTE** `Decline` handling and `customer` feedback.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



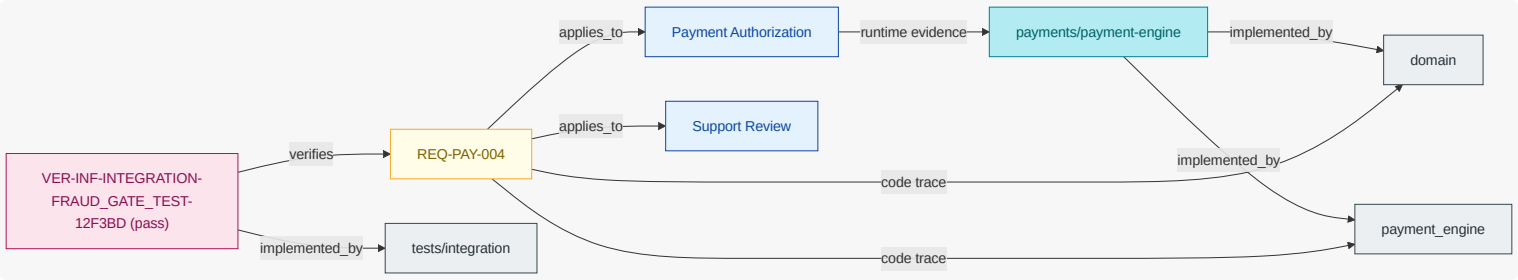
12.1.4. REQ-PAY-004

While `checkout is active`, when `payment authorization is requested` and `risk score is high` and `risk threshold is exceeded`, `the payments system` shall place the payment in `payment review is pending`.

> **NOTE** Escalation `flow` for high-risk `payments`.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



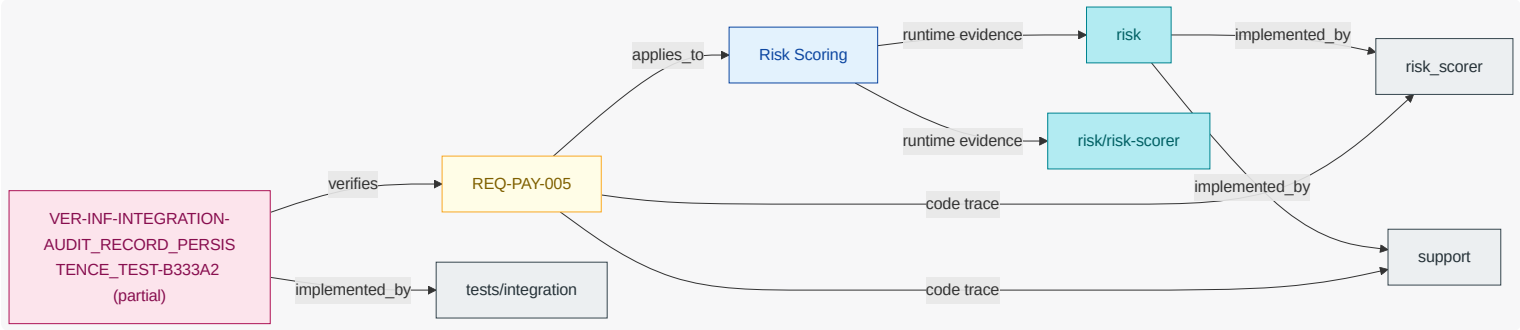
12.1.5. REQ-PAY-005

Where `fraud check is enabled`, when `payment authorization is requested`, `the payments system` shall persist an audit record with `payment id` plus `risk score`.

> **NOTE** Auditability for `authorization decisions`.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



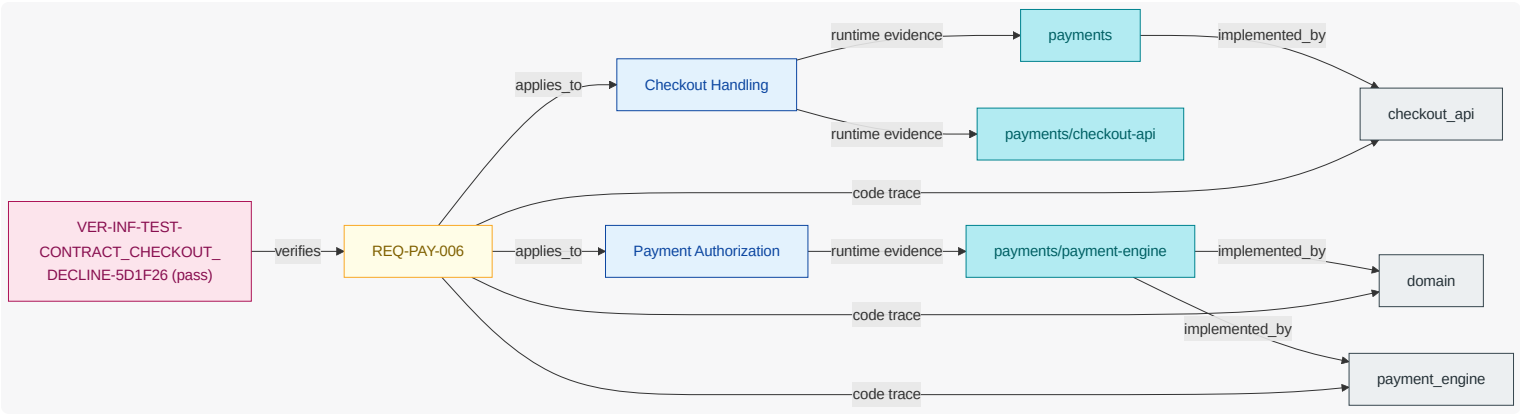
12.1.6. REQ-PAY-006

If `bank link is unavailable`, then `the payments system` shall present a temporary unavailable message to `customer` until `bank link is available`.

> **NOTE** Fallback behavior when external banking path is down.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



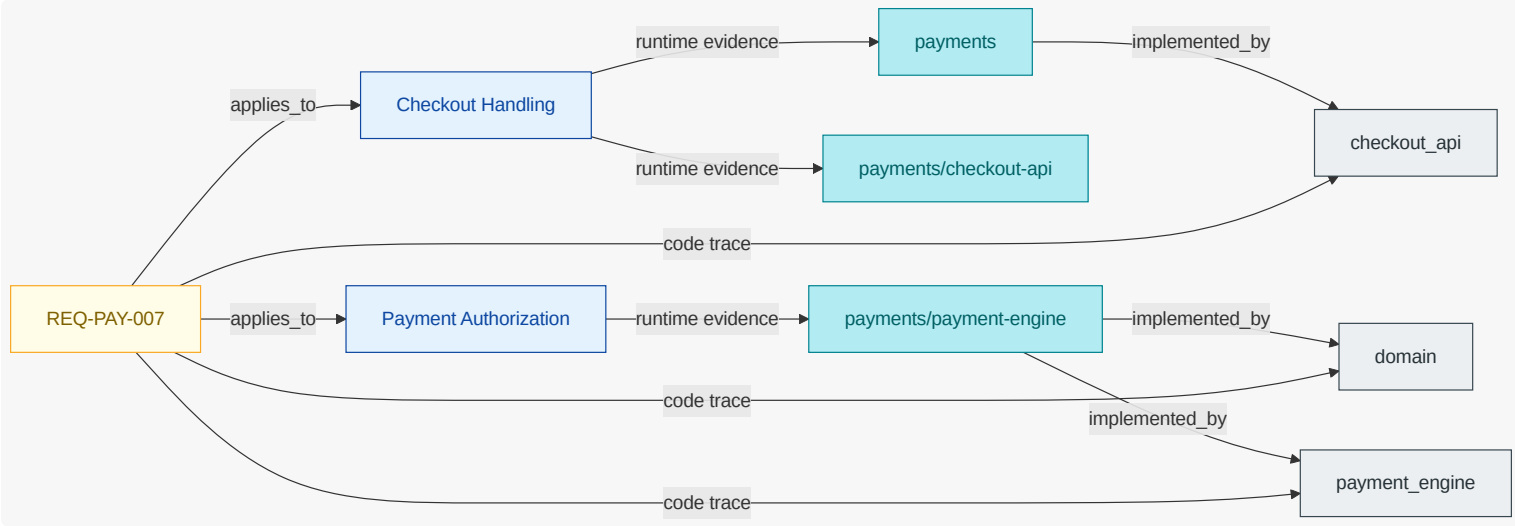
12.1.7. REQ-PAY-007

While `checkout is active`, when `payment authorization is requested` and `3DS verification is required`, `the payments system` shall request authorization from `bank gateway endpoint`.

> **NOTE** External authorization call contract.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



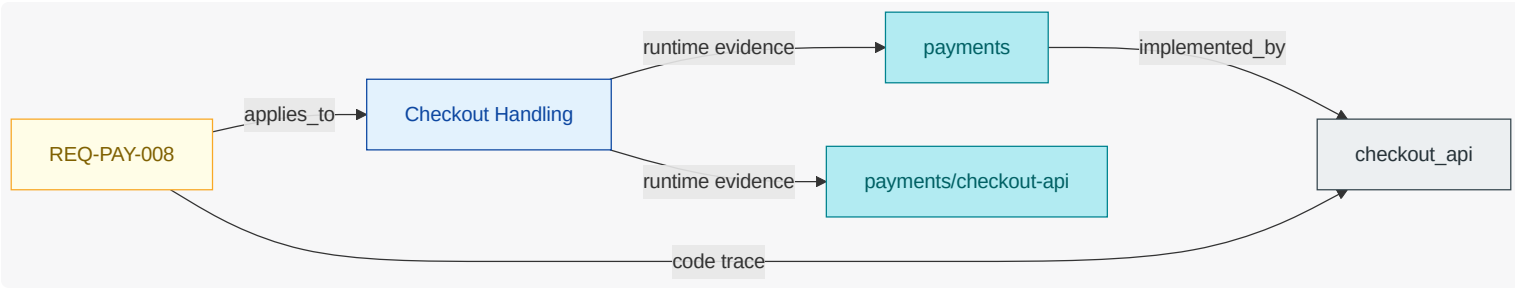
12.1.8. REQ-PAY-008

If `payment is declined`, then `the payments system` shall present a retry option to `customer`.

> **NOTE** `Customer` retry path `requirement` after `decline`.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



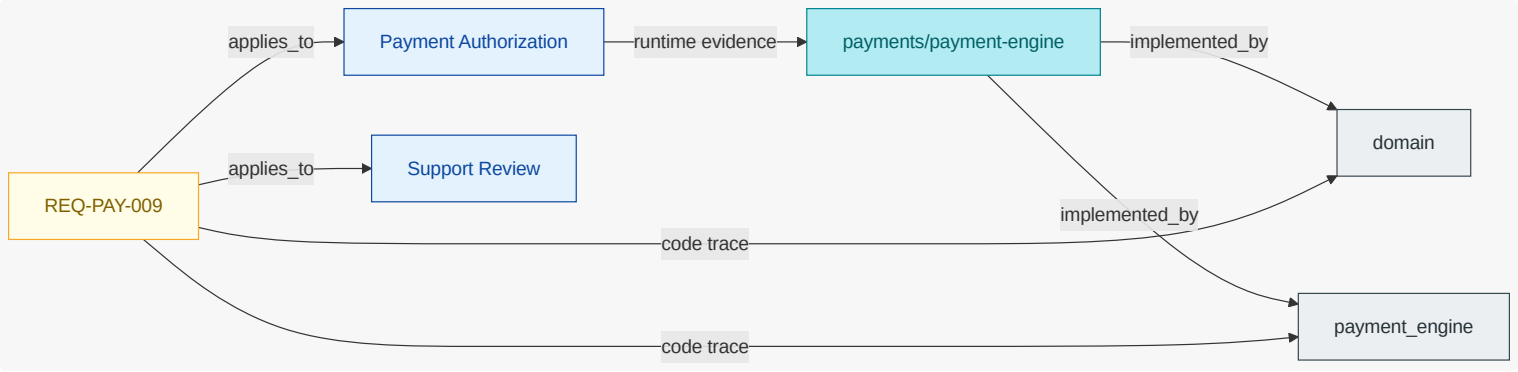
12.1.9. REQ-PAY-009

While `checkout is active`, when `payment authorization is requested` and `risk threshold is exceeded`, `the payments system` shall notify `platform operator` when `support agent is notified`.

> **NOTE** `Support` notification `requirement` for high-risk review.

Requirement-to-Unit-to-Evidence Coverage

What this diagram shows: this requirement-to-unit mapping extended with inferred runtime/code plus verification evidence attached to each requirement and unit.



12.2. Verification Inventory

What this section shows: verification checks inferred from test artifacts, linked to test code elements and mapped to verified requirements.

12.2.1. Authorized Checkout Flow

Field	Value
ID	`VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597`
Kind	e2e
Status	not-run
Verifies Requirements	`REQ-PAY-001`
Test Code Elements	tests/e2e/authorized_checkout_flow.yaml
Derived Functional Owners	`FU-CHECKOUT`, `FU-PAYMENT-AUTHORIZATION`
Result Summary	not-run:1
Evidence	tests/e2e/authorized_checkout_flow.yaml
Notes	executes `authorized checkout flow` from session start through `payment authorization` success

12.2.2. Audit Record Persistence Test

Field	Value
ID	`VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2`
Kind	integration
Status	partial
Verifies Requirements	`REQ-PAY-005`
Test Code Elements	tests/integration/audit_record_persistence_test.ts

Field	Value
Derived Functional Owners	`FU-RISK-SCORING`
Result Summary	partial:1
Evidence	test-results/audit-persistence-report.md, tests/integration/audit_record_persistence_test.ts
Notes	checks audit record persistence includes `payment id` and computed `risk score`

12.2.3. Fraud Gate Test

Field	Value
ID	`VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD`
Kind	integration
Status	pass
Verifies Requirements	`REQ-PAY-002`, `REQ-PAY-004`
Test Code Elements	tests/integration/fraud_gate_test.rs
Derived Functional Owners	`FU-PAYMENT-AUTHORIZATION`, `FU-RISK-SCORING`, `FU-SUPPORT-REVIEW`
Result Summary	pass:2
Evidence	test-results/integration-fraud-gate.json, tests/integration/fraud_gate_test.rs
Notes	validates fraud gate escalation and block decisions before authorization completion

12.2.4. Contract Checkout Decline

Field	Value
ID	`VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26`
Kind	test
Status	pass
Verifies Requirements	`REQ-PAY-003`, `REQ-PAY-006`
Test Code Elements	none
Derived Functional Owners	`FU-CHECKOUT`, `FU-PAYMENT-AUTHORIZATION`

Field	Value
Result Summary	pass:2
Evidence	test-results/contract-checkout-decline.json
Notes	Inferred from test result artifact.

13. Reference Index

13.1. Authored References

CTRL-PAYMENTS-IMAGE-DIGEST

category=supply-chain; description=Enforce immutable digest-pinned container image references.

Key	Value
Kind	Control
Name	Immutable Image Digests
Mentioned In	`Security View`

CTRL-PAYMENTS-SSO-MFA

category=identity-access; description=Enforce MFA and conditional access for privileged operations.

Key	Value
Kind	Control
Name	SSO MFA Enforcement
Mentioned In	`Architecture Intent View`, `Security View`, `State Lifecycle View`

DO-PAYMENTS-AUTH-DECISION

termRef=DATA-AUTH-DECISION; schemaRef=schemas/auth-decision.json; sensitivity=internal

Key	Value
Kind	Data Object
Name	Authorization Decision
Mentioned In	`Architecture Intent View`, `Communication View`, `Interaction Flow View`, `REQ-PAY-005`

DO-PAYMENTS-AUTH-REQUEST

termRef=DATA-PAYMENT-REQUEST; schemaRef=schemas/auth-request.json; sensitivity=confidential

Key	Value
Kind	Data Object
Name	Authorization Request

Key	Value
Mentioned In	`Architecture Intent View`, `Communication View`, `Interaction Flow View`

DO-PAYMENTS-REVIEW-TICKET

termRef=n/a; schemaRef=schemas/review-ticket.json; sensitivity=confidential

Key	Value
Kind	Data Object
Name	Manual Review Ticket
Mentioned In	`Architecture Intent View`, `Communication View`, `Interaction Flow View`

DO-PAYMENTS-RISK-SIGNAL

termRef=n/a; schemaRef=schemas/risk-signal.json; sensitivity=internal

Key	Value
Kind	Data Object
Name	Risk Signal
Mentioned In	`Architecture Intent View`, `Communication View`, `Interaction Flow View`

DEP-PAYMENTS-BANK-EDGE

environment=prod; region=us-east-1; account=external; cluster=bank-edge; namespace=partner; trustZone=external

Key	Value
Kind	Deployment Target
Name	Bank Edge Integration Zone
Mentioned In	`Architecture Intent View`, `Deployment View`

DEP-PAYMENTS-CLUSTER-PROD

environment=prod; region=us-east-1; account=shared-platform; cluster=payments-prod; namespace=payments; trustZone=app

Key	Value
Kind	Deployment Target
Name	Payments Cluster Production
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`

EVT-PAYMENTS-AUTH-COMPLETED

Authorization decision returned to checkout.

Key	Value
Kind	Event
Name	Authorization Completed
Mentioned In	`Architecture Intent View`, `State Lifecycle View`

EVT-PAYMENTS-AUTH-REQUESTED

Request sent to authorization orchestration.

Key	Value
Kind	Event
Name	Authorization Requested
Mentioned In	`Architecture Intent View`, `State Lifecycle View`

FLOW-CUSTOMER-CHECKOUT

entry=submit-payment; exits=show-outcome; steps=4

Key	Value
Kind	Flow
Name	Customer Checkout Authorization Flow
Mentioned In	`Interaction Flow View`

FLOW-PAYMENTS-MANUAL-REVIEW

entry=detect-high-risk; exits=complete-review; steps=3

Key	Value
Kind	Flow
Name	High-Risk Manual Review Flow
Mentioned In	`Interaction Flow View`

FLOW-CUSTOMER-CHECKOUT::authorize-payment

kind=system_action; ref=FU-PAYMENT-AUTHORIZATION; action=Request fraud score and external authorization; dataIn=normalized_payment_request; dataOut=authorization_decision; async=true

Key	Value
Kind	Flow Step
Name	Request fraud score and external authorization
Mentioned In	`Interaction Flow View`

FLOW-CUSTOMER-CHECKOUT::normalize-request

kind=system_action; ref=FU-CHECKOUT; action=Normalize and validate checkout payload; dataIn=; dataOut=normalized_payment_request; async=false

Key	Value
Kind	Flow Step
Name	Normalize and validate checkout payload
Mentioned In	`Interaction Flow View`

FLOW-CUSTOMER-CHECKOUT::show-outcome

kind=system_action; ref=FU-CHECKOUT; action=Return approval or decline outcome to customer; dataIn=authorization_decision; dataOut=; async=false

Key	Value
Kind	Flow Step
Name	Return approval or decline outcome to customer
Mentioned In	`Interaction Flow View`

FLOW-CUSTOMER-CHECKOUT::submit-payment

kind=user_action; ref=ACT-CUSTOMER; action=Customer submits checkout payment form; dataIn=payment_method, billing_address, amount; dataOut=; async=false

Key	Value
Kind	Flow Step
Name	Customer submits checkout payment form
Mentioned In	`Interaction Flow View`

FLOW-PAYMENTS-MANUAL-REVIEW::complete-review

kind=system_action; ref=FU-PAYMENT-AUTHORIZATION; action=Complete authorization using manual review outcome; dataIn=review_ticket; dataOut=; async=false

Key	Value
Kind	Flow Step
Name	Complete authorization using manual review outcome
Mentioned In	`Interaction Flow View`

FLOW-PAYMENTS-MANUAL-REVIEW::create-review-ticket

kind=system_action; ref=FU-SUPPORT-REVIEW; action=Create manual review ticket for support operations; dataIn=risk_signal; dataOut=review_ticket; async=false

Key	Value
Kind	Flow Step
Name	Create manual review ticket for support operations
Mentioned In	`Interaction Flow View`

FLOW-PAYMENTS-MANUAL-REVIEW::detect-high-risk

kind=system_action; ref=FU-RISK-SCORING; action=Detect high-risk signal and emit escalation trigger; dataIn=; dataOut=risk_signal; async=false

Key	Value
Kind	Flow Step
Name	Detect high-risk signal and emit escalation trigger
Mentioned In	`Interaction Flow View`

IF-PAYMENTS-BANK-AUTH

protocol=https; endpoint=/bank/authorize; schemaRef=schemas/bank-auth.json; owner=FU-PAYMENT-AUTHORIZATION

Key	Value
Kind	Interface
Name	Bank Authorization Interface
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Interaction Flow View`

IF-PAYMENTS-CHECKOUT-API

protocol=https; endpoint=/api/checkout; schemaRef=schemas/checkout-request.json; owner=FU-CHECKOUT

Key	Value
Kind	Interface
Name	Checkout API Interface
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Interaction Flow View`

IF-PAYMENTS-RISK-SCORE-API

protocol=https; endpoint=/risk/score; schemaRef=schemas/risk-score.json; owner=FU-RISK-SCORING

Key	Value
Kind	Interface
Name	Risk Score Interface
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Interaction Flow View`

REQ-PAY-001

While normal mode is active, when payment authorization is requested, the payments system shall create a payment session.

Key	Value
Kind	Requirement
Name	REQ-PAY-001
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Authorized Checkout Flow`

REQ-PAY-002

Where fraud check is enabled, when payment authorization is requested, the fraud detection system shall calculate a risk score before approval.

Key	Value
Kind	Requirement
Name	REQ-PAY-002
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Fraud Gate Test`

REQ-PAY-003

If payment is declined, then the payments system shall present a decline reason to customer.

Key	Value
Kind	Requirement
Name	REQ-PAY-003
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Contract Checkout Decline`

REQ-PAY-004

While checkout is active, when payment authorization is requested and risk score is high and risk threshold is exceeded, the payments system shall place the payment in payment review is pending.

Key	Value
Kind	Requirement
Name	REQ-PAY-004
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Fraud Gate Test`

REQ-PAY-005

Where fraud check is enabled, when payment authorization is requested, the payments system shall persist an audit record with payment id plus risk score.

Key	Value
Kind	Requirement
Name	REQ-PAY-005
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Audit Record Persistence Test`

REQ-PAY-006

If bank link is unavailable, then the payments system shall present a temporary unavailable message to customer until bank link is available.

Key	Value
Kind	Requirement
Name	REQ-PAY-006
Mentioned In	`Architecture Intent View`, `Deployment View`, `Interaction Flow View`, `Contract Checkout Decline`

REQ-PAY-007

While checkout is active, when payment authorization is requested and 3DS verification is required, the payments system shall request authorization from bank gateway endpoint.

Key	Value
Kind	Requirement
Name	REQ-PAY-007
Mentioned In	`Architecture Intent View`, `Deployment View`

REQ-PAY-008

If payment is declined, then the payments system shall present a retry option to customer.

Key	Value
Kind	Requirement
Name	REQ-PAY-008
Mentioned In	`Architecture Intent View`, `Deployment View`

REQ-PAY-009

While checkout is active, when payment authorization is requested and risk threshold is exceeded, the payments system shall notify platform operator when support agent is notified.

Key	Value
Kind	Requirement
Name	REQ-PAY-009
Mentioned In	`Architecture Intent View`, `Deployment View`

STATE-PAYMENTS-AUTHORIZED

Authorization decision approved.

Key	Value
Kind	State
Name	Payment Authorized
Mentioned In	`Architecture Intent View`, `State Lifecycle View`

STATE-PAYMENTS-CHECKOUT-RECEIVED

Payment request accepted at checkout boundary.

Key	Value
Kind	State
Name	Checkout Received
Mentioned In	`Architecture Intent View`, `State Lifecycle View`

TB-PAYMENTS-EXTERNAL-BANK

Separates internal payment authorization from external bank services.

Key	Value
Kind	Trust Boundary
Name	External Bank Boundary
Mentioned In	`Architecture Intent View`, `Security View`, `State Lifecycle View`

TB-PAYMENTS-PLATFORM-CONTROL

Separates app workloads from platform control-plane authority.

Key	Value
Kind	Trust Boundary
Name	Platform Control Boundary
Mentioned In	`Architecture Intent View`

13.2. Catalog References

ACT-CUSTOMER

End user submitting payment details during checkout.

Key	Value
Kind	Catalog Actor
Name	customer
Mentioned In	`Architecture Intent View`, `Communication View`, `Security View`, `Interaction Flow View`, `REQ-PAY-003`, `REQ-PAY-006`, `REQ-PAY-008`

ACT-PLATFORM-OPERATOR

Role responsible for GitOps and platform lifecycle operations.

Key	Value
Kind	Catalog Actor
Name	platform operator
Mentioned In	`Architecture Intent View`, `REQ-PAY-009`

ACT-SUPPORT

Operations role that investigates declines and high-risk review cases.

Key	Value
Kind	Catalog Actor
Name	support agent
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `REQ-PAY-009`

3DS required

Condition indicating strong customer authentication must be performed.

Key	Value
Kind	Catalog Alias
Name	3DS required
Mentioned In	n/a

auth

Coordinates authorization flow and bank gateway interaction.

Key	Value
Kind	Catalog Alias
Name	auth
Mentioned In	n/a

authorization requested

Event raised when checkout asks payment engine to authorize a transaction.

Key	Value
Kind	Catalog Alias
Name	authorization requested
Mentioned In	n/a

bank API

External banking endpoint used for card authorization and decline decisions.

Key	Value
Kind	Catalog Alias
Name	bank API
Mentioned In	n/a

bank link down

Event raised when payment engine cannot reach the external bank gateway.

Key	Value
Kind	Catalog Alias
Name	bank link down
Mentioned In	n/a

bank link up

Connectivity state showing bank gateway communication is currently healthy.

Key	Value
Kind	Catalog Alias
Name	bank link up
Mentioned In	n/a

buyer

End user submitting payment details during checkout.

Key	Value
Kind	Catalog Alias

Key	Value
Name	buyer
Mentioned In	n/a

callback replay

Replay of a valid callback to induce duplicate or unauthorized state transitions.

Key	Value
Kind	Catalog Alias
Name	callback replay
Mentioned In	n/a

checkout

Handles checkout session start and customer-facing payment messaging.

Key	Value
Kind	Catalog Alias
Name	checkout
Mentioned In	`Architecture Intent View`, `Deployment View`, `REQ-PAY-001`

checkout active

Processing state where the user is actively submitting or confirming payment.

Key	Value
Kind	Catalog Alias
Name	checkout active
Mentioned In	n/a

day1 provisioning

Provides cluster-level platform setup and environment bootstrap.

Key	Value
Kind	Catalog Alias
Name	day1 provisioning

Key	Value
Mentioned In	n/a

day2 operations

Operates declarative delivery and release management flows.

Key	Value
Kind	Catalog Alias
Name	day2 operations
Mentioned In	n/a

decline

Event indicating authorization was rejected and customer decline handling must execute.

Key	Value
Kind	Catalog Alias
Name	decline
Mentioned In	`REQ-PAY-003`, `REQ-PAY-008`

decline code

Structured reason code or message explaining why authorization failed.

Key	Value
Kind	Catalog Alias
Name	decline code
Mentioned In	n/a

decline reasons

Structured reason code or message explaining why authorization failed.

Key	Value
Kind	Catalog Alias
Name	decline reasons
Mentioned In	n/a

dependency compromise

Supply-chain compromise in runtime image or code dependency.

Key	Value
Kind	Catalog Alias
Name	dependency compromise
Mentioned In	n/a

fraud abuse

Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals.

Key	Value
Kind	Catalog Alias
Name	fraud abuse
Mentioned In	n/a

fraud check

Feature flag controlling whether risk scoring is required before authorization.

Key	Value
Kind	Catalog Alias
Name	fraud check
Mentioned In	n/a

fraud domain

Functional group for risk scoring and fraud review capabilities.

Key	Value
Kind	Catalog Alias
Name	fraud domain
Mentioned In	n/a

fraud scoring

Evaluates transaction risk and returns score classification.

Key	Value
Kind	Catalog Alias
Name	fraud scoring
Mentioned In	`Architecture Intent View`, `REQ-PAY-002`

helm

Runtime packaging and deployment substrate for workloads.

Key	Value
Kind	Catalog Alias
Name	helm
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Traceability View`

high risk

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Key	Value
Kind	Catalog Alias
Name	high risk
Mentioned In	n/a

high-risk classification

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Key	Value
Kind	Catalog Alias
Name	high-risk classification
Mentioned In	n/a

high-risk classifications

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Key	Value
Kind	Catalog Alias

Key	Value
Name	high-risk classifications
Mentioned In	n/a

malicious request

Crafted payload attempting to bypass validation or authorization logic.

Key	Value
Kind	Catalog Alias
Name	malicious request
Mentioned In	n/a

manual review

Supports manual review and escalation handling for risky transactions.

Key	Value
Kind	Catalog Alias
Name	manual review
Mentioned In	n/a

normal mode

Standard operating mode with full external dependency usage enabled.

Key	Value
Kind	Catalog Alias
Name	normal mode
Mentioned In	n/a

payment auth requested

Event raised when checkout asks payment engine to authorize a transaction.

Key	Value
Kind	Catalog Alias
Name	payment auth requested

Key	Value
Mentioned In	n/a

payment declined

Event indicating authorization was rejected and customer decline handling must execute.

Key	Value
Kind	Catalog Alias
Name	payment declined
Mentioned In	n/a

payment fraud

Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals.

Key	Value
Kind	Catalog Alias
Name	payment fraud
Mentioned In	n/a

payments domain

Functional group for checkout and authorization capabilities.

Key	Value
Kind	Catalog Alias
Name	payments domain
Mentioned In	n/a

platform engineering

Functional group for provisioning and platform operations capabilities.

Key	Value
Kind	Catalog Alias
Name	platform engineering

Key	Value
Mentioned In	n/a

platform ops

Role responsible for GitOps and platform lifecycle operations.

Key	Value
Kind	Catalog Alias
Name	platform ops
Mentioned In	n/a

postgres driver

Third-party library used for application data persistence connectivity.

Key	Value
Kind	Catalog Alias
Name	postgres driver
Mentioned In	n/a

review pending

Transaction state requiring manual support review before final disposition.

Key	Value
Kind	Catalog Alias
Name	review pending
Mentioned In	n/a

review state

Transaction state requiring manual support review before final disposition.

Key	Value
Kind	Catalog Alias
Name	review state
Mentioned In	n/a

risk above threshold

Condition indicating risk score crossed a policy threshold for escalation.

Key	Value
Kind	Catalog Alias
Name	risk above threshold
Mentioned In	n/a

standard operating mode

Standard operating mode with full external dependency usage enabled.

Key	Value
Kind	Catalog Alias
Name	standard operating mode
Mentioned In	n/a

support

Operations role that investigates declines and high-risk review cases.

Key	Value
Kind	Catalog Alias
Name	support
Mentioned In	n/a

support notified

Event indicating manual review notification was sent to support operations.

Key	Value
Kind	Catalog Alias
Name	support notified
Mentioned In	n/a

the fraud detection system

Fraud scoring and review subsystem participating in payment decisions.

Key	Value
Kind	Catalog Alias
Name	the fraud detection system
Mentioned In	`REQ-PAY-002`

the payments system

Payment processing system-of-interest covered by this architecture.

Key	Value
Kind	Catalog Alias
Name	the payments system
Mentioned In	`REQ-PAY-001`, `REQ-PAY-003`, `REQ-PAY-004`, `REQ-PAY-005`, `REQ-PAY-006`, `REQ-PAY-007`, `REQ-PAY-008`, `REQ-PAY-009`

AV-COMPROMISED-DEPENDENCY

Supply-chain compromise in runtime image or code dependency.

Key	Value
Kind	Catalog Attack Vector
Name	compromised dependency
Mentioned In	`Security View`

AV-FRAUDULENT-TRANSACTION-PATTERN

Coordinated abuse pattern using synthetic identities, stolen instruments, or velocity anomalies to obtain unauthorized approvals.

Key	Value
Kind	Catalog Attack Vector
Name	fraudulent transaction pattern
Mentioned In	`Security View`

AV-MALICIOUS-API-REQUEST

Crafted payload attempting to bypass validation or authorization logic.

Key	Value
Kind	Catalog Attack Vector
Name	malicious api request
Mentioned In	`Security View`

AV-REPLAYED-AUTH-CALLBACK

Replay of a valid callback to induce duplicate or unauthorized state transitions.

Key	Value
Kind	Catalog Attack Vector
Name	replayed authorization callback
Mentioned In	`Security View`

COND-3DS-REQUIRED

Condition indicating strong customer authentication must be performed.

Key	Value
Kind	Catalog Condition
Name	3DS verification is required
Mentioned In	`REQ-PAY-007`

COND-RISK-THRESHOLD-EXCEEDED

Condition indicating risk score crossed a policy threshold for escalation.

Key	Value
Kind	Catalog Condition
Name	risk threshold is exceeded
Mentioned In	`REQ-PAY-004`, `REQ-PAY-009`

DATA-DECLINE-REASON

Structured reason code or message explaining why authorization failed.

Key	Value
Kind	Catalog Data Term

Key	Value
Name	decline reason
Mentioned In	`REQ-PAY-003`

DATA-PAYMENT-ID

Stable identifier used to correlate a payment across services and records.

Key	Value
Kind	Catalog Data Term
Name	payment id
Mentioned In	`REQ-PAY-005`, `Audit Record Persistence Test`

DATA-RISK-SCORE

Numeric fraud score produced by risk evaluation logic.

Key	Value
Kind	Catalog Data Term
Name	risk score
Mentioned In	`REQ-PAY-002`, `REQ-PAY-005`, `Audit Record Persistence Test`

EVT-BANK-LINK-UNAVAILABLE

Event raised when payment engine cannot reach the external bank gateway.

Key	Value
Kind	Catalog Event
Name	bank link is unavailable
Mentioned In	`REQ-PAY-006`

EVT-PAYMENT-AUTH-REQUESTED

Event raised when checkout asks payment engine to authorize a transaction.

Key	Value
Kind	Catalog Event
Name	payment authorization is requested

Key	Value
Mentioned In	`REQ-PAY-001`, `REQ-PAY-002`, `REQ-PAY-004`, `REQ-PAY-005`, `REQ-PAY-007`, `REQ-PAY-009`

EVT-PAYMENT-DECLINED

Event indicating authorization was rejected and customer decline handling must execute.

Key	Value
Kind	Catalog Event
Name	payment is declined
Mentioned In	`REQ-PAY-003`, `REQ-PAY-008`

EVT-RISK-HIGH

Event emitted when computed fraud risk exceeds the configured high-risk threshold.

Key	Value
Kind	Catalog Event
Name	risk score is high
Mentioned In	`REQ-PAY-004`

EVT-SUPPORT-NOTIFIED

Event indicating manual review notification was sent to support operations.

Key	Value
Kind	Catalog Event
Name	support agent is notified
Mentioned In	`REQ-PAY-009`

FEAT-FRAUD-CHECK

Feature flag controlling whether risk scoring is required before authorization.

Key	Value
Kind	Catalog Feature
Name	fraud check is enabled
Mentioned In	`REQ-PAY-002`, `REQ-PAY-005`

FG-FRAUD

Functional group for risk scoring and fraud review capabilities.

Key	Value
Kind	Catalog Functional Group
Name	fraud evaluation
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`

FG-PAYMENTS

Functional group for checkout and authorization capabilities.

Key	Value
Kind	Catalog Functional Group
Name	payments
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`, `REQ-PAY-004`

FG-PLATFORM

Functional group for provisioning and platform operations capabilities.

Key	Value
Kind	Catalog Functional Group
Name	platform
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`, `Traceability View`

FU-CHECKOUT

Handles checkout session start and customer-facing payment messaging.

Key	Value
Kind	Catalog Functional Unit
Name	checkout handling
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Interaction Flow View`, `Authorized Checkout Flow`, `Contract Checkout Decline`

FU-CLUSTER-PROVISIONING

Provides cluster-level platform setup and environment bootstrap.

Key	Value
Kind	Catalog Functional Unit
Name	cluster provisioning
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`

FU-GITOPS-OPERATIONS

Operates declarative delivery and release management flows.

Key	Value
Kind	Catalog Functional Unit
Name	gitops operations
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`

FU-PAYMENT-AUTHORIZATION

Coordinates authorization flow and bank gateway interaction.

Key	Value
Kind	Catalog Functional Unit
Name	payment authorization
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Interaction Flow View`, `Authorized Checkout Flow`, `Fraud Gate Test`, `Contract Checkout Decline`

FU-RISK-SCORING

Evaluates transaction risk and returns score classification.

Key	Value
Kind	Catalog Functional Unit
Name	risk scoring
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Interaction Flow View`, `Audit Record Persistence Test`, `Fraud Gate Test`

FU-SUPPORT-REVIEW

Supports manual review and escalation handling for risky transactions.

Key	Value
Kind	Catalog Functional Unit
Name	support review
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `Interaction Flow View`, `Fraud Gate Test`

MODE-NORMAL

Standard operating mode with full external dependency usage enabled.

Key	Value
Kind	Catalog Mode
Name	normal mode is active
Mentioned In	`REQ-PAY-001`

REF-BANK-GATEWAY-ENDPOINT

External banking endpoint used for card authorization and decline decisions.

Key	Value
Kind	Catalog Referenced Element
Name	bank gateway endpoint
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`, `REQ-PAY-007`

REF-HELM-PLATFORM

Runtime packaging and deployment substrate for workloads.

Key	Value
Kind	Catalog Referenced Element
Name	helm platform
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`

REF-POSTGRES-DRIVER

Third-party library used for application data persistence connectivity.

Key	Value
Kind	Catalog Referenced Element
Name	postgresql client driver
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Security View`, `Traceability View`

STATE-BANK-LINK-UP

Connectivity state showing bank gateway communication is currently healthy.

Key	Value
Kind	Catalog State
Name	bank link is available
Mentioned In	`REQ-PAY-006`

STATE-CHECKOUT-ACTIVE

Processing state where the user is actively submitting or confirming payment.

Key	Value
Kind	Catalog State
Name	checkout is active
Mentioned In	`REQ-PAY-004`, `REQ-PAY-007`, `REQ-PAY-009`

STATE-REVIEW-PENDING

Transaction state requiring manual support review before final disposition.

Key	Value
Kind	Catalog State
Name	payment review is pending
Mentioned In	`REQ-PAY-004`

SYS-FRAUD-DETECTION-SYSTEM

Fraud scoring and review subsystem participating in payment decisions.

Key	Value
Kind	Catalog System
Name	fraud detection system
Mentioned In	n/a

SYS-PAYMENTS-SYSTEM

Payment processing system-of-interest covered by this architecture.

Key	Value
Kind	Catalog System
Name	payments system
Mentioned In	n/a

EM-ACTOR

A person or operational role that interacts with functional units.

Key	Value
Kind	Engineering Model Term
Name	actor
Mentioned In	`Architecture Intent View`, `Communication View`, `Interaction Flow View`

EM-ATTACK-VECTOR

A technical misuse or attack path that targets functional, referenced, or runtime elements.

Key	Value
Kind	Engineering Model Term
Name	attack vector
Mentioned In	n/a

EM-AUTHORED-MAPPING

An explicit relationship declared in architecture inputs between authored or referenced elements.

Key	Value
Kind	Engineering Model Term

Key	Value
Name	authored mapping
Mentioned In	`Communication View`

EM-CODE-ELEMENT

An inferred code structure or ownership element discovered from source trees and build metadata.

Key	Value
Kind	Engineering Model Term
Name	code element
Mentioned In	n/a

EM-CONTROL

An authored security, policy, or operational control used to constrain behavior and reduce risk.

Key	Value
Kind	Engineering Model Term
Name	control
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`, `State Lifecycle View`

EM-DATA-OBJECT

An authored data artifact or contract shape traced across flow, interface, and implementation boundaries.

Key	Value
Kind	Engineering Model Term
Name	data object
Mentioned In	n/a

EM-DEPLOYMENT-TARGET

An authored deployment destination such as environment, cluster, namespace, or registry scope.

Key	Value
Kind	Engineering Model Term
Name	deployment target

Key	Value
Mentioned In	n/a

EM-EVENT

An authored trigger signal that drives transitions, flow progress, or asynchronous outcomes.

Key	Value
Kind	Engineering Model Term
Name	event
Mentioned In	`Architecture Intent View`, `Communication View`, `State Lifecycle View`

EM-FLOW

An authored causal interaction sequence from user/system intent to outcome, represented as ordered steps.

Key	Value
Kind	Engineering Model Term
Name	flow
Mentioned In	`Architecture Intent View`, `Communication View`, `Security View`, `Interaction Flow View`, `REQ-PAY-004`

EM-FLOW-STEP

A single authored step in a flow that captures action, data in/out, references, and normal/error/async transitions.

Key	Value
Kind	Engineering Model Term
Name	flow step
Mentioned In	n/a

EM-FUNCTIONAL-GROUP

A major authored capability area that groups related functional units.

Key	Value
Kind	Engineering Model Term
Name	functional group

Key	Value
Mentioned In	n/a

EM-FUNCTIONAL-UNIT

An authored working unit inside a functional group that owns specific behavior.

Key	Value
Kind	Engineering Model Term
Name	functional unit
Mentioned In	`State Lifecycle View`

EM-INFERRED-MAPPING

A discovered relationship that links inferred runtime/code elements upward to authored design.

Key	Value
Kind	Engineering Model Term
Name	inferred mapping
Mentioned In	n/a

EM-INTERFACE

An authored interface boundary (for example API, channel, endpoint, or contract) owned by a functional unit.

Key	Value
Kind	Engineering Model Term
Name	interface
Mentioned In	`Architecture Intent View`, `Communication View`, `Deployment View`, `Interaction Flow View`

EM-REFERENCED-ELEMENT

An architecture-relevant external, platform, or third-party dependency represented by role.

Key	Value
Kind	Engineering Model Term
Name	referenced element
Mentioned In	n/a

EM-REQUIREMENT

A structured requirement statement that defines expected system behavior and maps to functional ownership.

Key	Value
Kind	Engineering Model Term
Name	requirement
Mentioned In	`Architecture Intent View`, `Deployment View`, `Traceability View`, `Interaction Flow View`, `REQ-PAY-008`, `REQ-PAY-009`

EM-RUNTIME-ELEMENT

An inferred runtime realization element discovered from infrastructure and deployment sources.

Key	Value
Kind	Engineering Model Term
Name	runtime element
Mentioned In	n/a

EM-STATE

An authored lifecycle state used to model transition behavior, guards, and event-driven progression.

Key	Value
Kind	Engineering Model Term
Name	state
Mentioned In	`Architecture Intent View`, `Deployment View`, `State Lifecycle View`

EM-TRUST-BOUNDARY

An authored trust separation zone that marks policy, identity, or security control boundaries.

Key	Value
Kind	Engineering Model Term
Name	trust boundary
Mentioned In	`Security View`

EM-UPWARD-LINKING

Rule where runtime and code elements point to stable functional groups/units; authored architecture does not depend on inferred IDs.

Key	Value
Kind	Engineering Model Term
Name	upward linking
Mentioned In	n/a

EM-VERIFICATION-CHECK

An inferred or authored verification artifact that validates requirement behavior with test evidence.

Key	Value
Kind	Engineering Model Term
Name	verification check
Mentioned In	n/a

13.3. Inferred Runtime References

payments

provisions the shared EKS control plane that hosts payments and risk workloads

Key	Value
Kind	Runtime cluster
Owner	FU-CLUSTER-PROVISIONING
Source	examples/payments-engineering-sample/infra/terraform/main.tf
Mentioned In	n/a

flux-system/payments-sample-source

tracks the desired repository state that drives Flux reconciliation for payments workloads

Key	Value
Kind	Runtime gitrepository
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/flux/sources/payments-source.yaml

Key	Value
Mentioned In	`Deployment View`

payments/checkout-api

exposes checkout ingress and forwards normalized payment requests to authorization flow

Key	Value
Kind	Runtime helmrelease
Owner	FU-CHECKOUT
Source	examples/payments-engineering-sample/infra/flux/releases/checkout-api.yaml
Mentioned In	`Deployment View`, `Security View`

payments/payment-engine

executes payment authorization decisions and coordinates fraud checks plus bank callbacks

Key	Value
Kind	Runtime helmrelease
Owner	FU-PAYMENT-AUTHORIZATION
Source	examples/payments-engineering-sample/infra/flux/releases/payment-engine.yaml
Mentioned In	`Deployment View`, `Security View`

risk/risk-scorer

computes fraud risk score signals and persists evidence used by authorization decisions

Key	Value
Kind	Runtime helmrelease
Owner	FU-RISK-SCORING
Source	examples/payments-engineering-sample/infra/flux/releases/risk-scorer.yaml
Mentioned In	`Deployment View`, `Security View`

flux-system/payments-apps

applies and reconciles HelmRelease definitions for checkout, payment, and risk services

Key	Value
Kind	Runtime kustomization
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/flux/kustomizations/payments-apps.yaml
Mentioned In	`Deployment View`

flux_system

control-plane namespace where Flux GitOps operators reconcile deployment state

Key	Value
Kind	Runtime namespace
Owner	FU-GITOPS-OPERATIONS
Source	examples/payments-engineering-sample/infra/terraform/main.tf
Mentioned In	`Deployment View`

payments

runtime namespace boundary for checkout and payment authorization workloads

Key	Value
Kind	Runtime namespace
Owner	FU-CHECKOUT
Source	examples/payments-engineering-sample/infra/terraform/main.tf
Mentioned In	n/a

risk

runtime namespace boundary for fraud scoring and review support workloads

Key	Value
Kind	Runtime namespace
Owner	FU-RISK-SCORING
Source	examples/payments-engineering-sample/infra/terraform/main.tf
Mentioned In	`Architecture Intent View`, `Deployment View`, `Security View`

13.4. Inferred Code References

gopkg.in/yaml.v3

Inferred library_external dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_external
Owner	FU-CHECKOUT
Source	checkout_api.go
Mentioned In	n/a

serde_json::json

Inferred library_external dependency owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs.

Key	Value
Kind	Code library_external
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs
Mentioned In	n/a

zod

Inferred library_external dependency owned by FU-RISK-SCORING from risk_scorer.ts.

Key	Value
Kind	Code library_external
Owner	FU-RISK-SCORING
Source	risk_scorer.ts
Mentioned In	n/a

./support/audit_envelope

Inferred library_first_party dependency owned by FU-RISK-SCORING from risk_scorer.ts.

Key	Value
Kind	Code library_first_party

Key	Value
Owner	FU-RISK-SCORING
Source	risk_scorer.ts
Mentioned In	n/a

crate::domain::events::PaymentEvent

Inferred library_first_party dependency owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs.

Key	Value
Kind	Code library_first_party
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs
Mentioned In	n/a

github.com/labeth/engineering-model-go/validate

Inferred library_first_party dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_first_party
Owner	FU-CHECKOUT
Source	checkout_api.go
Mentioned In	n/a

fmt

Inferred library_stdlib dependency owned by FU-CHECKOUT from checkout_api.go.

Key	Value
Kind	Code library_stdlib
Owner	FU-CHECKOUT
Source	checkout_api.go
Mentioned In	n/a

checkout_api.go

accepts checkout requests and forwards payment authorization flow input

Key	Value
Kind	Code source_file
Owner	FU-CHECKOUT
Source	checkout_api.go
Mentioned In	`Deployment View`, `Security View`

domain/events.rs

defines payment domain events used across authorization and risk paths

Key	Value
Kind	Code source_file
Owner	FU-PAYMENT-AUTHORIZATION
Source	domain/events.rs
Mentioned In	n/a

payment_engine.rs

executes authorization decisions and persists payment-side audit records

Key	Value
Kind	Code source_file
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs
Mentioned In	`Deployment View`, `Security View`

risk_scorer.ts

computes fraud risk scores and decides review or decline outcomes

Key	Value
Kind	Code source_file
Owner	FU-RISK-SCORING
Source	risk_scorer.ts

Key	Value
Mentioned In	`Deployment View`, `Security View`

support/audit_envelope.ts

formats support-review audit envelopes for traceability workflows

Key	Value
Kind	Code source_file
Owner	FU-RISK-SCORING
Source	support/audit_envelope.ts
Mentioned In	n/a

CODE-AUTHORIZEPAYMENT

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:11.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:11
Mentioned In	n/a

CODE-BUILDREVIEWREASON

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:50.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:50
Mentioned In	n/a

CODE-CALCULATERISKSCORE

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:8.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:8
Mentioned In	n/a

CODE-CLASSIFYRISK

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:21.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:21
Mentioned In	n/a

CODE-CREATEAUDITRECORD

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:37.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:37
Mentioned In	n/a

CODE-ENRICHAUDITMETADATA

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:45.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:45
Mentioned In	n/a

CODE-HANDLEBANKLINKUNAVAILABLE

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:34.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:34
Mentioned In	n/a

CODE-HANDLEBANKUNAVAILABLE

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:32.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:32
Mentioned In	n/a

CODE-ISHIGHRISK

Inferred code symbol owned by FU-RISK-SCORING from risk_scorer.ts:29.

Key	Value
Kind	Code symbol
Owner	FU-RISK-SCORING
Source	risk_scorer.ts:29
Mentioned In	n/a

CODE-NOTIFYSUPPORT

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:52.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION

Key	Value
Source	payment_engine.rs:52
Mentioned In	n/a

CODE-PERSISTAUDITRECORD

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:60.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:60
Mentioned In	n/a

CODE-PLACEINREVIEW

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:44.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:44
Mentioned In	n/a

CODE-REQUESTBANKAUTHORIZATION

Inferred code symbol owned by FU-PAYMENT-AUTHORIZATION from payment_engine.rs:26.

Key	Value
Kind	Code symbol
Owner	FU-PAYMENT-AUTHORIZATION
Source	payment_engine.rs:26
Mentioned In	n/a

CODE-SHOWDECLINEREASON

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:37.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:37
Mentioned In	n/a

CODE-SHOWRETRYOPTION

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:42.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:42
Mentioned In	n/a

CODE-SHOWTEMPORARYUNAVAILABLE

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:47.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:47
Mentioned In	n/a

CODE-STARTSESSION

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:16.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:16
Mentioned In	n/a

CODE-SUBMITAUTHORIZATIONREQUEST

Inferred code symbol owned by FU-CHECKOUT from checkout_api.go:27.

Key	Value
Kind	Code symbol
Owner	FU-CHECKOUT
Source	checkout_api.go:27
Mentioned In	n/a

13.5. Verification References

VER-INF-E2E-AUTHORIZED_CHECKOUT_FLOW-8C6597

executes authorized checkout flow from session start through payment authorization success

Key	Value
Kind	e2e
Status	not-run
Source	tests/e2e/authorized_checkout_flow.yaml
Name	Authorized Checkout Flow
Mentioned In	`Interaction Flow View`, `Authorized Checkout Flow`

VER-INF-INTEGRATION-AUDIT_RECORD_PERSISTENCE_TEST-B333A2

checks audit record persistence includes payment id and computed risk score

Key	Value
Kind	integration
Status	partial
Source	test-results/audit-persistence-report.md (+1)
Name	Audit Record Persistence Test
Mentioned In	`Interaction Flow View`, `Audit Record Persistence Test`

VER-INF-INTEGRATION-FRAUD_GATE_TEST-12F3BD

validates fraud gate escalation and block decisions before authorization completion

Key	Value
Kind	integration
Status	pass
Source	test-results/integration-fraud-gate.json (+1)
Name	Fraud Gate Test
Mentioned In	`Interaction Flow View`, `Fraud Gate Test`

VER-INF-TEST-CONTRACT_CHECKOUT_DECLINE-5D1F26

Inferred from test result artifact.

Key	Value
Kind	test
Status	pass
Source	test-results/contract-checkout-decline.json
Name	Contract Checkout Decline
Mentioned In	`Interaction Flow View`, `Contract Checkout Decline`